

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH**



Nhóm 8

Hồ Văn Đô - 2351010049

Lê Khánh Duy - 2351010032

Trần Quang Huy - 2351010079

Nguyễn Nhật Đăng - 2351010045

Nguyễn Hoàng Tấn Phúc - 2351010164

**EmoNeXt: an Adapted ConvNeXt
for facial Emotion Recognition**

ĐỒ ÁN MÔN HỌC

MÔN: MÁY HỌC

TP. HỒ CHÍ MINH, 2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

MỤC LỤC

Chương 1. Giới thiệu.....	4
1.1. Tổng quan bài toán nhận dạng cảm xúc khuôn mặt (FER).....	4
1.1.1. Tầm quan trọng của biểu cảm khuôn mặt.....	4
1.1.2. Ứng dụng của học sâu (Deep Learning) vào hệ thống FER.....	5
1.2. Hạn chế của các phương pháp hiện tại.....	5
1.3. Phương pháp đề xuất.....	6
1.4. Đóng góp của đề tài.....	6
Chương 2. Các nghiên cứu liên quan.....	6
2.1. Các mô hình học sâu áp dụng trên tập dữ liệu FER2013.....	6
2.1.1. Phương pháp kết hợp đặc trưng học sâu và thủ công.....	7
2.1.2. Phương pháp cải tiến cơ chế chú ý và hàm mất mát.....	7
Chương 3. Phương pháp đề xuất.....	7
3.1. Tổng quan kiến trúc mạng EmoNeXt.....	7
3.2. Mạng biến đổi không gian (Spatial Transformer Networks - STN).....	13
3.3. Kiến trúc khối ConvNeXt.....	17
3.4. Cơ chế Squeeze-and-Excitation (SE).....	26
3.5. Thành phần điều chuẩn tự chú ý (Self-Attention Regularization Term) kết hợp hàm mất mát Cross-Entropy.....	28
3.6. Các tinh chỉnh của nhóm.....	30
3.6.1. Sửa lỗi trong cơ chế Self-Attention.....	30
3.6.2. Cân chỉnh hàm mất mát.....	33
3.6.3. Cải tiến quy trình huấn luyện.....	33
3.6.4. Sửa lỗi trong pipeline đánh giá.....	34
3.6.5. Ghi chú về tính nhất quán val/test.....	35
Chương 4. Kết quả và thực nghiệm.....	35
4.1. Tập dữ liệu FER2013.....	35
4.1.1. Tổng quan và phân chia dữ liệu.....	35
4.1.2. Vấn đề mất cân bằng lớp.....	36
4.2. Cấu hình huấn luyện và Chuyển giao học tập.....	36
4.3. Phân tích nguyên nhân và quá trình khắc phục.....	37
4.3.1. Hiện tượng mô hình không hội tụ.....	37
4.3.2. Điều chỉnh siêu tham số và Learning Rate (Đạt mức ~67%).....	38
4.4. Đánh giá hiệu suất mô hình tinh chỉnh của nhóm.....	38
4.4.1. Phân tích kết quả Validation.....	38
4.4.2. Kết quả đánh giá trên tập Test.....	40
4.4.3. So sánh định lượng với các mô hình SOTA.....	42

Chương 5. Kết luận.....	43
5.1. Tóm tắt hiệu quả của EmoNeXt.....	43
5.2. Hướng phát triển trong tương lai.....	44

CHƯƠNG 1. GIỚI THIỆU

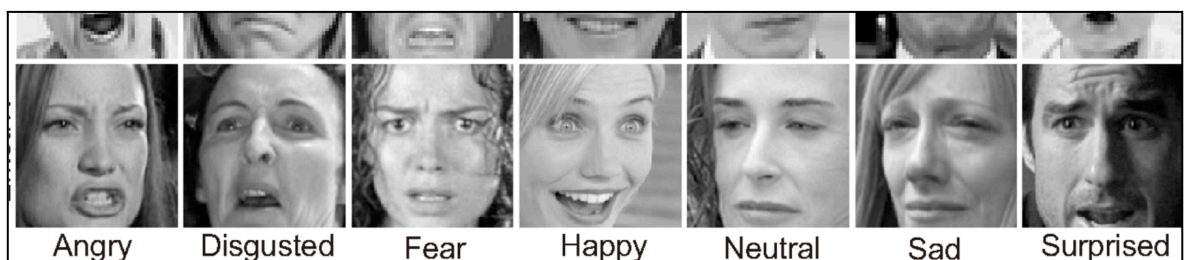
1.1. Tổng quan bài toán nhận dạng cảm xúc khuôn mặt (FER)

Nhận dạng cảm xúc khuôn mặt (Facial Expression Recognition - FER) tự động từ hình ảnh tĩnh là một thách thức lớn trong lĩnh vực thị giác máy tính. Các phương pháp FER truyền thống thường tuân theo quy trình 2 bước: (1) phân tích và định nghĩa các đặc trưng khuôn mặt (dựa trên hình học hoặc diện mạo bề mặt), (2) sử dụng các đặc trưng này để suy luận thông qua các bộ phân loại độc lập. Tuy nhiên, do hai bước này được thiết kế tách rời, mô hình không thể học đặc trưng một cách tối ưu theo mục tiêu phân loại, từ đó làm hạn chế hiệu suất tổng thể.

Với sự phát triển của học sâu, các phương pháp dựa trên mạng nơ-ron tích chập (CNN) cho phép học đặc trưng một cách end-to-end, giúp cải thiện đáng kể hiệu quả của bài toán FER. Tuy nhiên, đối với các tập dữ liệu thực tế như FER2013, bài toán vẫn còn nhiều thách thức do sự mất cân bằng giữa các lớp cảm xúc, cũng như các biến thiên phức tạp về ánh sáng, góc nhìn, biểu cảm tinh vi và nhiễu trong ảnh.

1.1.1. Tầm quan trọng của biểu cảm khuôn mặt

Biểu cảm khuôn mặt là phương tiện giao tiếp phi ngôn ngữ cốt lõi, giúp truyền tải lượng lớn thông tin và đa dạng cảm xúc mà không cần dùng đến lời nói. Theo nghiên cứu của Albert Mehrabian, ngôn ngữ cơ thể (đặc biệt là biểu cảm khuôn mặt) chiếm tỷ lệ áp đảo lên tới 55% hiệu quả giao tiếp, vượt xa giọng điệu (38%) và ngôn từ (chỉ 7%). Chính vì vai trò quan trọng này, việc thiết kế máy tính để mô hình hóa và nhận thức tự động biểu cảm của con người là một hướng đi hoàn toàn có cơ sở khoa học và vô cùng cần thiết.



Hình 1. Các hình đại diện cho cảm xúc

1.1.2. Ứng dụng của học sâu (Deep Learning) vào hệ thống FER

Ứng dụng	Trong hệ thống FER	Lợi ích và ưu điểm
Trích xuất đặc trưng	Sử dụng các mạng nơ-ron để tự động học các biểu diễn đặc trưng mang tính phân biệt từ dữ liệu hình ảnh thô.	Giảm thiểu sự phụ thuộc vào các mô hình hình học thủ công và các kỹ thuật tiền xử lý phức tạp.
Xử lý biến thiên không gian	Tích hợp mạng biến đổi không gian để tự động học và áp dụng các phép biến đổi hình học lên ảnh đầu vào.	Giúp mô hình bền vững trước các thay đổi về tỷ lệ, góc xoay và vị trí tịnh tiến của khuôn mặt.
Cơ chế chú ý và Điều chỉnh	Các kỹ thuật Self-attention kết hợp với các hàm mất mát để diễn đặc trưng ở chiều không gian thấp.	Khuyến khích sự cân bằng về tầm quan trọng giữa các đặc trưng và cải thiện khả năng tổng quát hóa của mô hình.
Chiến lược huấn luyện nâng cao	Sử dụng các kỹ thuật hỗ trợ như bộ tối ưu hóa Adam, tăng cường dữ liệu, làm mượt nhãn và huấn luyện với độ chính xác hỗn hợp	Tăng tốc độ hội tụ, tối ưu hóa tài nguyên phần cứng và hạn chế hiện tượng quá khớp (overfitting).

1.2. Hạn chế của các phương pháp hiện tại

Trong bài toán FER, các phương pháp hiện tại vẫn đang đối mặt với 3 thách thức cốt lõi: (1) sự biến thiên về không gian như góc xoay, vị trí tịnh tiến và sự che khuất làm giảm đáng kể tính bền vững của các mạng CNN truyền thống, (2) sự mất cân bằng dữ liệu trong các tập dữ liệu thực tế khiến mô hình dễ bị sai lệch (bias) về phía các lớp chiếm đa số, gây ảnh hưởng đến độ chính xác tổng quát, (3) các kiến trúc dựa trên Transformer mang lại hiệu năng cao nhưng lại yêu cầu khối lượng dữ liệu khổng lồ để huấn luyện và tiêu hao nhiều tài nguyên tính toán.

1.3. Phương pháp đề xuất

Để giải quyết các thách thức hiện tại, nghiên cứu đề xuất EmoNeXt - Khung học sâu được xây dựng trên kiến trúc xương sống ConvNeXt. Mô hình kế thừa và tích hợp các ưu điểm của Vision Transformer (ViT) vào trong một mạng nơ-ron tích chập (CNN) thuần túy. EmoNeXt sử dụng các phép tích chập tách biệt theo chiều sâu (depthwise convolution) với kích thước kernel lớn nhằm giảm thiểu chi phí tính toán đồng thời mở rộng trường thụ cảm (tầm nhìn mà 1 nơ-ron sử dụng để nhìn và xử lý dữ liệu đầu vào). Đặc biệt, kiến trúc này tích hợp Mạng biến đổi không gian (STN) để tập trung vào các vùng chứa nhiều đặc trưng trên khuôn mặt, cùng các khối Squeeze-and-Excitation (SE) để nắm bắt sự phụ thuộc giữa các kênh (channel-wise dependencies) và thành phần điều chuẩn tự chú ý (self-attention regularization term) kết hợp với hàm mất mát Cross-Entropy truyền thống, nhằm khuyến khích mô hình tạo ra các vector đặc trưng cô đọng.

1.4. Đóng góp của đề tài

Phát hiện và chẩn đoán mô hình gốc: Nhóm đã tái tạo thực nghiệm từ code gốc của tác giả, phát hiện hiện tượng mô hình không hội tụ với cấu hình mặc định. Qua quá trình phân tích, xác định nguyên nhân liên quan đến learning rate, scheduler chưa tối ưu với fine-tuning. Sau đó, nhóm điều chỉnh theo gợi ý từ cộng đồng, đạt test accuracy là 66.9451%. Vẫn chưa đạt được kì vọng như bài báo công bố, nhóm thực hiện đối chiếu chi tiết giữa mô tả trong paper và code thực nghiệm, phát hiện 4 vấn đề kỹ thuật trong cơ chế Self-Attention: (1) attention scaling có sự nhầm lẫn trong công thức, (2) Global Average Pooling được áp dụng trước attention dẫn đến attention matrix vô nghĩa về mặt semantic trong máy học, (3) attention output bị bỏ qua hoàn toàn trong classification head, (4) SA loss tính mean chưa khớp với chiều so với định nghĩa trong paper.

Cải tiến và thực nghiệm kiểm chứng: Dựa trên các phân tích, nhóm thực hiện 10 điều chỉnh có chủ đích bao gồm tinh chỉnh kiến trúc, cân chỉnh hàm mất mát, thay thế scheduler, và hiệu chỉnh pipeline đánh giá. Mô hình sau cải tiến đạt **val accuracy 72.4561%** và **test accuracy 72.5416%**, cải thiện đáng kể so với mức 66.9451% đạt được với code gốc đã được điều chỉnh LR và mức chênh lệch gần hơn với kết quả bài báo.

CHƯƠNG 2. CÁC NGHIÊN CỨU LIÊN QUAN

2.1. Các mô hình học sâu áp dụng trên tập dữ liệu FER2013

Học sâu (Deep Learning) đã tạo nên bước ngoặt trong bài toán Nhận diện cảm xúc khuôn mặt (FER) nhờ khả năng trích xuất đặc trưng tự động qua các cấu trúc phi

tuyến phức tạp. Sau thành công của AlexNet (2012), hàng loạt kiến trúc như CNN, DBN, DAE hay các mô hình pre-trained (VGG-Face, GoogleNet) đã được áp dụng trên tập FER2013 để tối ưu hóa độ chính xác. Tổng quan các nghiên cứu hiện nay tập trung vào hai hướng chính: (1) Kết hợp đặc trưng học sâu và thủ công, (2) Cải tiến cơ chế chú ý và hàm mất mát.

2.1.1. Phương pháp kết hợp đặc trưng học sâu và thủ công

Nhiều nghiên cứu tận dụng đồng thời sức mạnh của CNN với các đặc trưng thủ công như BOVW, LBP hay CLBP để cải thiện hiệu suất. Tuy nhiên, hướng đi này thường đòi hỏi các bước tiền xử lý phức tạp và thiết kế kiến trúc phân tách, làm mất đi tính nguyên khối (end-to-end) của hệ thống.

2.1.2. Phương pháp cải tiến cơ chế chú ý và hàm mất mát

Để tăng tính phân biệt cho đặc trưng, các nghiên cứu hiện đại tập trung vào:

- Cơ chế chú ý: Sử dụng các mô-đun như Local multi-Head Channel (LHC) để định vị các vùng khuôn mặt quan trọng.
- Hàm mất mát: Đề xuất các hàm mới như Ad-Corre kết hợp cùng Cross-Entropy để thắt chặt khoảng cách giữa các vector đặc trưng cùng loại.

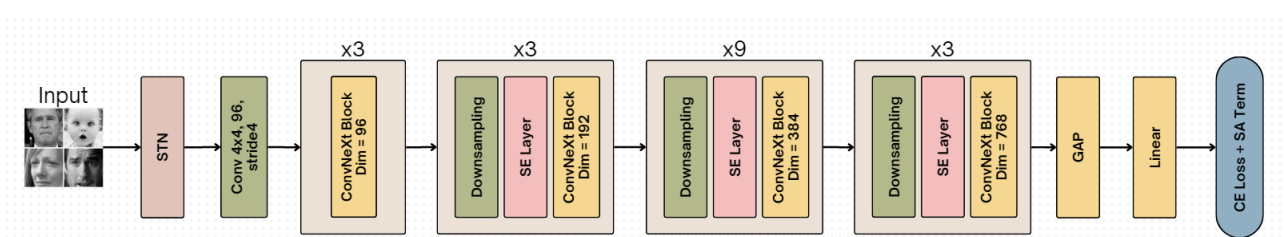
Kế thừa và tối ưu: EmoNeXt tiếp nối xu hướng này bằng cách tích hợp khối Squeeze-and-Excitation (SE) để điều chỉnh trọng số kênh và áp dụng thành phần điều chuẩn tự chú ý (Self-Attention Regularization). Cách tiếp cận này giúp mô hình đạt được hiệu quả vượt trội nhưng vẫn giữ được sự tinh gọn so với các phương pháp tiền nhiệm.

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT

3.1. Tổng quan kiến trúc mạng EmoNeXt

EmoNeXt là một mô hình học sâu được đề xuất cho bài toán nhận diện biểu cảm khuôn mặt (FER), dựa trên nền tảng là kiến trúc ConvNeXt – Một mô hình thuần túy tích chập được thiết kế để cạnh tranh với Vision Transformer trong khi vẫn giữ được sự đơn giản và hiệu quả của CNN. Tuy nhiên, ConvNeXt nguyên bản không được tối ưu riêng cho tác vụ FER, vốn đòi hỏi khả năng xử lý các biến dạng không gian của khuôn mặt, tầm quan trọng khác nhau giữa các vùng trên mặt và sự phụ thuộc tinh tế giữa các kênh đặc trưng.

Để giải quyết các hạn chế này, EmoNeXt giới thiệu 3 cải tiến chính được tích hợp có hệ thống vào kiến trúc ConvNeXt, tạo thành một mô hình thống nhất và mạnh mẽ cho FER.



Hình 2: Mô hình kiến trúc của EmoNeXt-Tiny

Như được trình bày trong Hình 2, kiến trúc EmoNeXt mô hình hóa quá trình nhận dạng thông qua luồng xử lý dữ liệu gồm 4 giai đoạn cốt lõi sau:

- **Tiền xử lý và phân mảnh dữ liệu (Preprocessing & Patchify):** Quá trình bắt đầu với Mạng biến đổi không gian (STN) nhằm tự động học và áp dụng phép biến đổi không gian Affine, giúp ảnh đầu vào đạt được sự bất biến đối với các thay đổi về tỷ lệ, góc xoay và tịnh tiến. Ảnh sau khi chuẩn hóa không đi thẳng vào các khối xử lý sâu mà được đưa qua một module phân mảnh (Patchify module). Module này sử dụng một lớp tích chập không chồng lấn (kernel 4x4, stride 4) kết hợp với Layer Normalization (LN) để hạ độ phân giải không gian và chuyển đổi ảnh thành bản đồ đặc trưng cơ sở (kích thước 56x56 với 96 kênh).
- **Trích xuất đặc trưng đa giai đoạn với cơ chế chú ý (SE):** Bản đồ đặc trưng cơ sở tiếp tục đi qua 4 giai đoạn xử lý chính (với tỷ lệ block tương ứng: 3-3-9-3 đối với phiên bản Tiny). Điểm cải tiến của EmoNeXt nằm ở việc tái cấu trúc các giai đoạn từ thứ 2 trở đi. Cụ thể, sau bước hạ lấy mẫu không gian (Downsampling) ở mỗi giai đoạn, một khối Squeeze-and-Excitation (SE) được chèn vào trước các khối ConvNeXt. Lớp SE này đóng vai trò kiểm tra và thiết lập lại độ chính xác của trọng số theo kênh, đảm bảo các đặc trưng quan trọng được nhấn mạnh trước khi mạng ConvNeXt thực hiện các phép tích chập phức tạp.
- **Tổng hợp đặc trưng:** Sau khi trải qua 4 giai đoạn, đầu ra được đưa qua một lớp gộp trung bình toàn cục (GAP) kết hợp với Layer Normalization để nén toàn bộ thông tin không gian thành một vector đặc trưng biểu diễn một chiều thống nhất.
- **Phân loại và tối ưu hóa:** Cuối cùng, vector đã qua tinh chỉnh được đưa vào một tầng tuyến tính đầy đủ (Linear layer) đóng vai trò là bộ phân loại để dự đoán xác suất trên 7 lớp cảm xúc. Để tối ưu, kiến trúc sử dụng một hàm mất mát tổng hợp, trong đó thành phần điều chuẩn Tự chú ý (Self-Attention Regularization Term) được kết hợp với hàm Cross-Entropy truyền thống. Sự kết hợp này nhằm cực tiểu hóa phương sai của các trọng số chú ý, khuyến khích mô hình tạo ra các vector đặc trưng có mức độ cân bằng và độ chính xác phân loại cao.

Lớp EmoNeXt trong file models.py gốc của tác giả hiện thực hóa toàn bộ pipeline trên:

```

142  ✓ class EmoNext(nn.Module):
143  ✓     def __init__(
144         self,
145         in_chans=3,
146         num_classes=1000,
147         depths=None,
148         dims=None,
149         drop_path_rate=0.0,
150         layer_scale_init_value=1e-6,
151     ):
152         super().__init__()
153
154         if dims is None:
155             dims = [96, 192, 384, 768]
156         if depths is None:
157             depths = [3, 3, 9, 3]
158
159         # Spatial transformer localization-network
160         self.localization = nn.Sequential(
161             nn.Conv2d(3, 8, kernel_size=7),
162             nn.BatchNorm2d(8),
163             nn.MaxPool2d(2, stride=2),
164             nn.ReLU(True),
165             nn.Conv2d(8, 10, kernel_size=5),
166             nn.BatchNorm2d(10),
167             nn.MaxPool2d(2, stride=2),
168             nn.ReLU(True),
169         )
170
171         # Regressor for the 3 * 2 affine matrix
172         self.fc_loc = nn.Sequential(
173             nn.Linear(10 * 52 * 52, 32), nn.ReLU(True), nn.Linear(32, 3 * 2)
174         )

```

Thành phần STN được triển khai thông qua 2 modules con: mạng định vị (self.localization) và bộ điều chỉnh tham số Affine (self.fc_loc). Mạng định vị có nhiệm vụ trích xuất các đặc trưng không gian từ ảnh đầu vào, trong khi bộ điều chỉnh ánh xạ các đặc trưng này thành 6 tham số của ma trận biến đổi Affine 2×3 .

```

176         self.downsample_layers = (
177             nn.ModuleList()
178         ) # stem and 3 intermediate downsampling conv layers
179         stem = nn.Sequential(
180             nn.Conv2d(in_chans, dims[0], kernel_size=4, stride=4),
181             LayerNorm(dims[0], eps=1e-6, data_format="channels_first"),
182         )
183         self.downsample_layers.append(stem)
184         for i in range(3):
185             downsample_layer = nn.Sequential(
186                 LayerNorm(dims[i], eps=1e-6, data_format="channels_first"),
187                 nn.Conv2d(dims[i], dims[i + 1], kernel_size=2, stride=2),
188                 SELayer(dims[i + 1]),
189             )
190             self.downsample_layers.append(downsample_layer)

```

Trước khi đi vào các giai đoạn trích xuất đặc trưng chính (stages), ảnh đầu vào sau khi qua bộ biến đổi STN sẽ được xử lý bởi tầng Stem. Tầng này được triển khai như một lớp Convolution với `kernel_size = 4` và `stride = 4`. Việc sử dụng stride bằng với kernel size giúp mô hình chia ảnh thành các mảnh (patches) không chồng lấn lên nhau. Điều này giúp giảm nhanh kích thước không gian của ảnh ngay từ bước đầu, giúp mô hình tập trung vào các đặc trưng cấp cao và giảm chi phí tính toán cho các lớp sau. Theo sau là một lớp LN để chuẩn hóa dữ liệu trước khi đi vào các khối ConvNeXt.

Các tầng Downsampling đóng vai trò chuyển tiếp giữa các giai đoạn, vừa giảm kích thước không gian của đặc trưng vừa tăng số lượng kênh. Đặc biệt, EmoNeXt tích hợp khối SE vào ngay sau mỗi tầng Downsampling.

Khác với mô tả trong phần tổng quan của bài báo, mã nguồn cho thấy SE block được đặt bên trong tầng Downsampling, áp dụng ngay sau khi tăng số kênh và trước khi đặc trưng được đưa vào stage kế tiếp. Cách bố trí này cho phép SE điều chỉnh trọng số của các kênh vừa được mở rộng, tạo điều kiện cho các block ConvNeXt trong stage tiếp theo hoạt động trên các đặc trưng đã được tái cân bằng.

```

192         self.stages = (
193             nn.ModuleList()
194         ) # 4 feature resolution stages, each consisting of multiple residual blocks
195         dp_rates = [x.item() for x in torch.linspace(0, drop_path_rate, sum(depths))]
196         cur = 0
197         for i in range(4):
198             stage = nn.Sequential(
199                 *[
200                     Block(
201                         dim=dims[i],
202                         drop_path=dp_rates[cur + j],
203                         layer_scale_init_value=layer_scale_init_value,
204                     )
205                     for j in range(depths[i])
206                 ]
207             )
208             self.stages.append(stage)
209             cur += depths[i]

```

Mỗi stage trong EmoNeXt bao gồm một chuỗi tuần tự các ConvNeXt block. Số lượng block mỗi stage được xác định bởi tham số depths và tỷ lệ Drop rate (Stochastic Depth) tăng dần qua các block. Tỷ lệ drop tăng tuyến tính từ 0 ở block đầu tiên đến drop_path_rate ở block cuối cùng của mạng. Chiến lược này dựa trên quan sát rằng các block ở giai đoạn sớm thường trích xuất các đặc trưng cơ bản và thiết yếu, do đó cần được giữ lại với xác suất cao. Ngược lại, các block ở giai đoạn muộn có thể bị bỏ qua ngẫu nhiên trong quá trình huấn luyện mà không ảnh hưởng đáng kể đến chất lượng của mô hình, đồng thời giúp chống overfitting.

```

211         self.norm = nn.LayerNorm(dims[-1], eps=1e-6) # final norm layer
212         self.attention = DotProductSelfAttention(dims[-1])
213         self.head = nn.Linear(dims[-1], num_classes)
214
215         for m in self.modules():
216             if isinstance(m, (nn.Conv2d, nn.Linear)):
217                 nn.init.trunc_normal_(m.weight, std=0.02)
218                 if m.bias is not None:
219                     nn.init.zeros_(m.bias)
220
221         self.fc_loc[2].weight.data.zero_()
222         self.fc_loc[2].bias.data.copy_(
223             torch.tensor([1, 0, 0, 0, 1, 0], dtype=torch.float)
224         )

```

Phần cuối của lớp EmoNeXt bao gồm các thành phần quan trọng: tầng chuẩn hóa cuối, cơ chế tự chú ý và bộ phân loại. Bên cạnh đó, việc khởi tạo tham số đóng vai trò then chốt trong sự hội tụ của mô hình.

```

237 ✓ def forward_features(self, x):
238     for i in range(4):
239         x = self.downsample_layers[i](x)
240         x = self.stages[i](x)
241     return self.norm(
242         x.mean([-2, -1])
243     ) # global average pooling, (N, C, H, W) -> (N, C)
244

```

Phương thức `forward_features` đóng vai trò là bộ khung chính của quá trình trích xuất đặc trưng, điều phối luồng dữ liệu qua 4 giai đoạn (stages) liên tiếp của mạng. 4 giai đoạn của mạng được thực thi tuần tự thông qua vòng lặp `for i in range(4)`. Mỗi giai đoạn bao gồm hai bước xử lý liên tiếp: Hạ bậc không gian và tăng số chiều kênh (Downsampling), Chuỗi các khối ConvNeXt để trích xuất đặc trưng (Stages).

Ngoài ra, còn có sự thay đổi so với tác giả qua việc áp dụng Global Average Pooling (GAP) sớm, trả về vector (B,C) - Khiến attention sau đó chỉ hoạt động trên batch dimension (vô nghĩa về mặt semantic). Nên đổi lại dùng cách giữ lại spatial dimensions (B, C, 7, 7), attention có thể hoạt động trên 49 spatial tokens, cho phép các vùng khuôn mặt (mắt, miệng, mày) chú ý lẫn nhau - Phù hợp hơn cho FER.

```

245 ✓ def forward(self, x, labels=None):
246     x = self.stn(x)
247     x = self.forward_features(x)
248     _, weights = self.attention(x)
249     logits = self.head(x)
250
251     if labels is not None:
252         mean_attention_weight = torch.mean(weights)
253         attention_loss = torch.mean((weights - mean_attention_weight) ** 2)
254
255         loss = F.cross_entropy(logits, labels, label_smoothing=0.2) + attention_loss
256         return torch.argmax(logits, dim=1), logits, loss
257
258     return torch.argmax(logits, dim=1), logits

```

Phương thức `forward` định nghĩa toàn bộ pipeline của mô hình, từ ảnh đầu vào đến dự đoán cuối cùng. Nó tích hợp tất cả các thành phần đã được mô tả và hỗ trợ cả hai chế độ huấn luyện và suy luận.

Quá trình xử lý được tổ chức thành bốn bước tuần tự: (1) Chuẩn hóa không gian khuôn mặt thông qua phép biến đổi Affine học được từ STN; (2) Trích xuất đặc trưng đa tỷ lệ qua 4 giai đoạn ConvNeXt + SE; (3) Tính toán trọng số tự chú ý `self.attention`; (4) Ánh xạ tuyến tính sang 7 lớp cảm xúc. Khi tham số `labels` được

cung cấp (khác None), mô hình hoạt động ở chế độ huấn luyện và thực hiện tính toán mất mát.

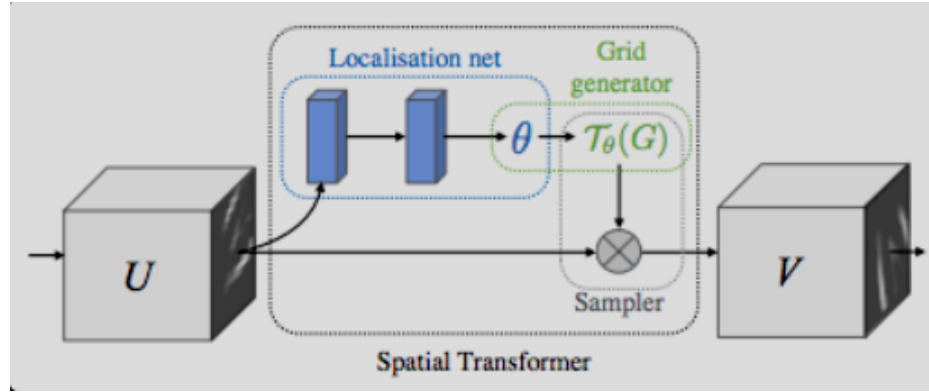
Tương tự như ConvNeXt, EmoNeXt cũng được xây dựng với 5 biến thể khác nhau dựa trên độ sâu (số block trong mỗi giai đoạn) và độ rộng (số kênh đặc trưng) của mạng. Các cấu hình này ảnh hưởng trực tiếp đến số lượng tham số, chi phí tính toán và hiệu năng của mô hình.

Biến thể	Kích thước kênh (C)	Số Block (B)
EmoNeXt-T	(96, 192, 384, 768)	(3, 3, 9, 3)
EmoNeXt-S	(96, 192, 384, 768)	(3, 3, 27, 3)
EmoNeXt-B	(128, 256, 512, 1024)	(3, 3, 27, 3)
EmoNeXt-L	(192, 384, 768, 1536)	(3, 3, 27, 3)
EmoNeXt-XL	(256, 512, 1024, 2048)	(3, 3, 27, 3)

3.2. Mạng biến đổi không gian (Spatial Transformer Networks - STN)

Tổng quan về STN:

Trong lĩnh vực học máy và thị giác máy tính, STN là một module có thể tích hợp trực tiếp vào các mạng nơ-ron đặc biệt là CNN, nó đóng vai trò quan trọng trong việc tối ưu hóa dữ liệu: Thay vì xử lý thủ công, STN tự động học ma trận biến đổi theta để căn chỉnh và chuẩn hóa khuôn mặt về trạng thái lý tưởng, loại bỏ các tác động tiêu cực từ góc nghiêng hay sự lệch hướng. Với các bộ dữ liệu nhỏ như FER2013, STN giúp mô hình tập trung vào các vùng biểu cảm quan trọng (mắt, mũi, miệng) thông qua các phép biến đổi hình học như phóng to hoặc cắt vùng, đồng thời tạo ra tính bất biến trước các biến dạng xoay hay co giãn. Module này hoạt động thông qua 3 thành phần chính liên kết với nhau trong một quy trình thống nhất: Localization Network, Grid Generator và Sampler. Toàn bộ quá trình này diễn ra một cách liên tục và có thể học được thông qua lan truyền ngược.



Hình 3: Cấu trúc của STN



Hình 4: Ảnh khuôn mặt sau khi được STN nắn chỉnh

In shape trước STN khi dữ liệu đi vào có dạng [Batch_size, Channels, Height, Width], nhưng lúc này ảnh có thể bị nghiêng, lệch hoặc khuôn mặt quá nhỏ so với khung hình. Sau khi qua STN thì đầu ra của ảnh vẫn cùng dạng nhưng các pixel đã được sắp xếp lại, khuôn mặt từ chỗ bị nghiêng được xoay thẳng

1. Localization Net:

Localization Net đóng vai trò là khối học tham số biến đổi, có nhiệm vụ xác định cách thức biến đổi ảnh đầu vào để đưa về dạng phù hợp nhất cho các bước xử lý tiếp theo. Cụ thể, Localization Network nhận ảnh làm đầu vào và thông qua các lớp tích chập kết hợp với các lớp fully connected, mạng sẽ dự đoán các tham số của phép biến đổi hình học, thường được biểu diễn dưới dạng ma trận theta θ .

Ma trận theta thường tương ứng với các phép biến đổi affine như tịnh tiến, xoay, co giãn hoặc biến dạng nhẹ. Nhờ đó, Localization Network có khả năng học cách căn chỉnh khuôn mặt một cách tối ưu dựa trên dữ liệu huấn luyện, thay vì phải sử dụng các phương pháp tiền xử lý thủ công. Quá trình học này hoàn toàn được thực hiện thông qua cơ chế lan truyền ngược (backpropagation), giúp mạng tự động tối ưu để

phục vụ tốt nhất cho nhiệm vụ chính. Đặc biệt, Localization Network không trực tiếp tạo ra ảnh đã biến đổi mà chỉ cung cấp các tham số biến đổi.

Đầu ra của Localization Network là ma trận biến đổi theta:

$$\begin{bmatrix} x_s \\ y_s \\ 1 \end{bmatrix} = \begin{bmatrix} \theta_{11} & \theta_{12} & \theta_{13} \\ \theta_{21} & \theta_{22} & \theta_{23} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_t \\ y_t \\ 1 \end{bmatrix}$$

Trong đó:

(x_t, y_t) : tọa độ điểm trên ảnh đầu ra

(x_s, y_s) : tọa độ tương ứng trên ảnh đầu vào

2. Grid Generator

Sau khi Localization Network dự đoán được ma trận theta, Grid Generator sẽ sử dụng ma trận này để tạo ra một tập hợp các điểm tọa độ trong không gian ảnh đầu vào. Grid Generator xây dựng một lưới chuẩn trên ảnh đầu ra, sau đó áp dụng phép biến đổi hình học để ánh xạ các điểm này ngược về ảnh đầu vào. Grid Generator thực hiện việc biến đổi toàn bộ lưới điểm dựa trên ma trận theta thay vì biến đổi trực tiếp từng điểm ảnh.

Có một vấn đề là các tọa độ (x_s, y_s) thường là giá trị thực, do ảnh hưởng của các phép biến đổi như xoay hoặc co giãn. Vì vậy các giá trị này chưa thể dùng trực tiếp để lấy pixel mà sẽ được chuyển sang bước tiếp theo là Sampler, tại đây sẽ thực hiện nội suy song tuyến để xác định giá trị pixel tương ứng.

3. Sampler

Sampler là bước cuối cùng trong quá trình biến đổi không gian, có nhiệm vụ tạo ra ảnh đầu ra thực sự dựa trên lưới tọa độ được cung cấp bởi Grid Generator. Sau khi Grid Generator sinh ra các tọa độ ánh xạ (x_s, y_s) tương ứng với từng điểm trên ảnh đầu ra, Sampler sẽ sử dụng các tọa độ này để lấy giá trị pixel từ ảnh đầu vào. Tuy nhiên, do các tọa độ (x_s, y_s) thường là số thực nên việc lấy giá trị trực tiếp là không khả thi. Vì vậy, Sampler sử dụng phương pháp nội suy để ước lượng giá trị pixel tại các vị trí này.

Phương pháp nội suy phổ biến nhất được sử dụng là nội suy song tuyến tính, qua đó giá trị của một điểm ảnh đầu ra được tính dựa trên tổ hợp có trọng số của 4 điểm ảnh lân cận gần nhất trong ảnh đầu vào. Được biểu diễn bằng công thức tổng quát như sau:

$$V(x_t, y_t) = \sum_i \sum_j U(i, j) \cdot \max(0, 1 - |x_s - i|) \cdot \max(0, 1 - |y_s - j|)$$

Trong đó:

$U(i, j)$: giá trị pixel tại vị trí (i, j) của ảnh đầu vào.

$V(x_t, y_t)$: giá trị pixel tại vị trí (x_t, y_t) trên ảnh đầu ra

x_s, y_s : tọa độ lấy mẫu trên ảnh nguồn.

$\max(0, 1 - |x_s - i|)$ và $\max(0, 1 - |y_s - j|)$: các trọng số nội suy song tuyến tính.

Ưu điểm của phương pháp nội suy là tính khả vi, tức là cho phép gradient được lan truyền ngược qua Sampler trong quá trình huấn luyện. Điều này đảm bảo rằng toàn bộ STN từ bước lấy mẫu cho đến Localization Network đều có thể được tối ưu hóa đồng thời thông qua backpropagation.

Backpropagation: Cơ chế cốt lõi cho phép mạng nơ-ron học và tối ưu hóa các tham số. Quá trình này được thực hiện sau bước lan truyền thuận, khi mô hình đã tạo ra dự đoán và tính toán được hàm mất mát dựa trên sự khác biệt giữa giá trị dự đoán và nhãn thực tế.

Cụ thể, Backpropagation tiến hành tính toán gradient của hàm mất mát đối với từng tham số trong mạng bằng cách áp dụng quy tắc dây chuyền (Chain Rule) của đạo hàm. Gradient này phản ánh mức độ ảnh hưởng của mỗi trọng số đến sai số đầu ra. Sau khi xác định được các gradient, mô hình sử dụng các thuật toán tối ưu để cập nhật trọng số, với mục tiêu là làm giảm giá trị của hàm mất mát qua từng vòng lặp huấn luyện. Trong quá trình lan truyền ngược sẽ tính gradient của hàm mất mát theo tọa độ lấy mẫu x_s và y_s

$$\frac{\delta L}{\delta x_s} = \sum_i \sum_j \frac{\delta L}{\delta V(x_t, y_t)} * U(i, j) * \frac{\delta}{\delta x_s} [\max(0, 1 - |x_s - i|) * \max(0, 1 - |y_s - j|)]$$

$$\frac{\delta L}{\delta y_s} = \sum_i \sum_j \frac{\delta L}{\delta V(x_t, y_t)} * U(i, j) * \frac{\delta}{\delta y_s} [\max(0, 1 - |y_s - j|)] * \max(0, 1 - |x_s - i|]$$

Trong đó:

$\frac{\delta L}{\delta V(x_t, y_t)}$: gradient của loss theo pixel đầu ra tại vị trí (x_t, y_t)

Sau quá trình lan truyền ngược, gradient của hàm mất mát được truyền qua Spatial Transformer để cập nhật các trọng số của Localization Network. Thông qua các

thuật toán tối ưu như Gradient Descent, mạng dần học được các tham số affine tối ưu, giúp căn chỉnh đặc trưng đầu vào phù hợp hơn cho quá trình học và phân loại ở các tầng tiếp theo.

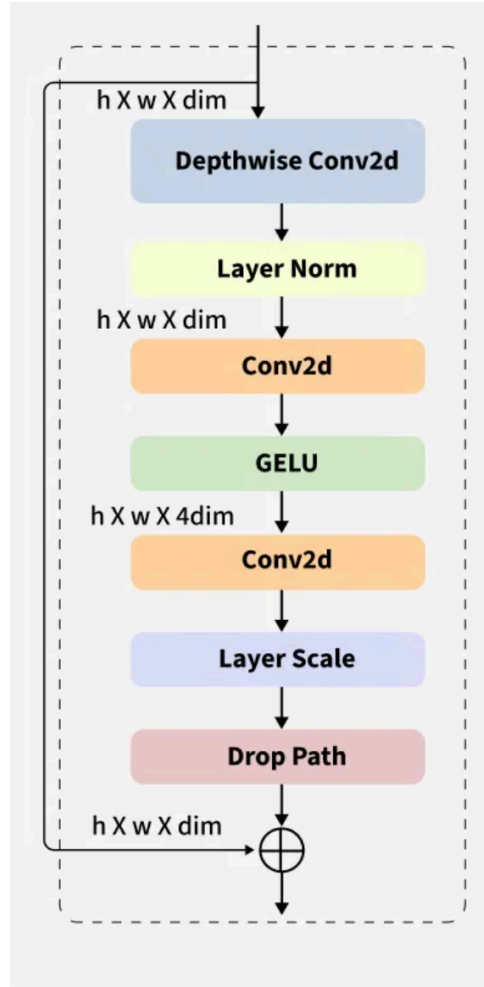
3.3. Kiến trúc khối ConvNeXt

Tổng quan ConvNeXt:

ConvNeXt được tổ chức thành 4 stage xử lý đặc trưng với số lượng kênh tăng dần. Với phiên bản Tiny - Được sử dụng trong EmoNeXt có cấu hình cụ thể là:

Stage	Số kênh (C)	Số block (B)
Stage 1	96	3
Stage 2	192	3
Stage 3	384	9
Stage 4	768	3

Thiết kế Convnext Block: Đơn vị cơ bản của ConvNeXt là ConvNeXt Block, được thiết kế theo dạng inverted bottleneck với 4 thành phần chính: Depthwise Convolution, Layer Normalization, Inverted Bottleneck với GELU, Layer Scale và Stochastic Depth.



Hình 5: Kiến trúc bên trong ConvNext Block

Toàn bộ luồng xử lý trong một ConvNeXt Block có thể viết như sau:

$$x' = x + \text{StochasticDepth}(\gamma \cdot \text{Pointwise}(\text{GELU}(\text{Pointwise}(\text{LN}(\text{DWConv}7 \times 7(x)))))$$

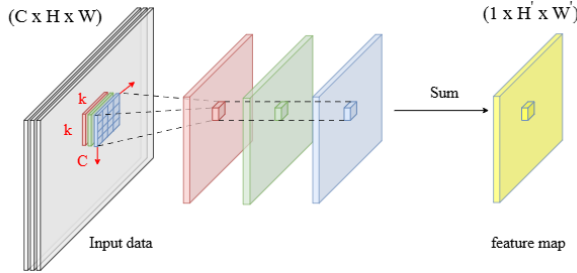
Các thành phần cốt lõi:

1. Depthwise Convolution kernel 7×7 , padding = 3:

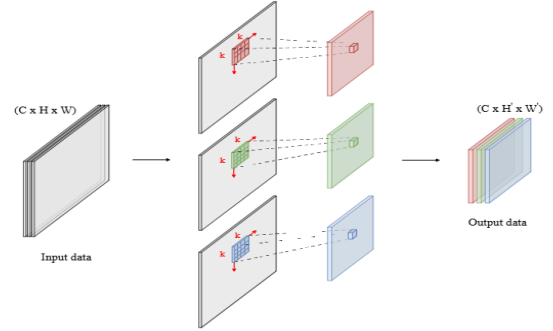
Trong các mạng tích chập truyền thống, mỗi phép tích chập tiêu chuẩn xử lý đồng thời 2 loại thông tin: đặc trưng không gian trên từng kênh và mối quan hệ giữa các kênh với nhau. Sự đan xen này tạo ra chi phí tính toán lớn và hạn chế khả năng mở rộng mô hình. ConvNeXt giải quyết vấn đề trên bằng cách tách 2 luồng thông tin thành 2 phép toán độc lập: Depthwise Convolution xử lý đặc trưng không gian riêng từng kênh (mỗi kênh có một kernel riêng, không có trao đổi thông tin chéo giữa các kênh), trong khi Pointwise Convolution (tích chập 1×1) đảm nhận việc tổng hợp thông tin xuyên kênh. Sự phân tách này giảm đáng kể chi phí tính toán so với tích

chập tiêu chuẩn, đồng thời cho phép mở rộng kernel size lên 7×7 mà không làm bùng nổ số lượng tham số

Hình 6: Minh họa Convolution tiêu chuẩn



Hình 7: Minh họa Depthwise Convolution



Trong tích chập tiêu chuẩn, một kernel có kích thước k thực chất là một khối 3D có kích thước $C \times k \times k$, trong đó chiều sâu luôn bằng với số kênh của dữ liệu đầu vào. Khi khối kernel này trượt qua toàn bộ không gian dữ liệu, số lượng phép tính cần thiết cho một bản đồ đặc trưng đầu ra là $C \times k^2 \times W \times H$. Nếu mô hình yêu cầu n bản đồ đặc trưng ở đầu ra, tổng chi phí tính toán:

$$FLOPs_{Standard} = C \times k \times k \times W \times H = C \times k^2 \times n \times H \times W$$

Ngược lại, Depthwise Convolution đơn giản hóa quy trình này bằng cách xử lý độc lập từng kênh đầu vào thông qua một kernel 2D kích thước $k \times k$. Với tổng cộng C kênh đầu vào, ta cần C kernel tương ứng, dẫn đến tổng chi phí tính toán cho toàn bộ quá trình này là:

$$FLOPs_{Depthwise} = C \times k \times k \times W \times H = C \times k^2 \times H \times W$$

So với tích chập tiêu chuẩn, Depthwise Convolution đã giảm chi phí đi n lần. Tuy nhiên, bước này chỉ trả về C kênh đầu ra (bằng đầu vào) và chưa thiết lập tương quan thông tin giữa các kênh. Để tạo ra n kênh đầu ra mới và tổng hợp thông tin liên kênh, hệ thống cần tiếp tục qua Pointwise Convolution.

2. Layer Normalization (LN):

Input: $(N, C, H, W) \rightarrow$ Output: (N, C, H, W) - Tensor không đổi kích thước

Sau Depthwise Conv, các giá trị activation có thể có phân phối không ổn định. Layer Normalization - Một kỹ thuật phổ biến trong Transformer đưa các giá trị này

về phân phối ổn định hơn trong các tình huống batch size nhỏ và fine-tuning. LN hoạt động nguyên tắc "mỗi vị trí không gian là một vector độc lập", tức là với ảnh kích thước $H \times W$, mỗi vị trí không gian (h, w) có một vector 96 chiều (theo C channel). LN normalize từng vector này một cách độc lập:

Với pixel tại vị trí (h, w) : vector $x = [x_1, x_2, \dots, x_C] \in \mathbb{R}^C$

B1: $\mu = (1/C) \sum_i x_i$ (Mean - Trung bình của vector)

B2: $\sigma^2 = (1/C) \sum_i (x_i - \mu)^2$ (Variance - Phương sai của vector)

B3: $\hat{x}_i = (x_i - \mu) / \sqrt{(\sigma^2 + \epsilon)}$ (Normalize - Chuẩn hóa)

B4: $y_i = \gamma_i \cdot \hat{x}_i + \beta_i$ (Scale & Shift learnable - Điều hành phân phối tối ưu nhất mà mạng cần)

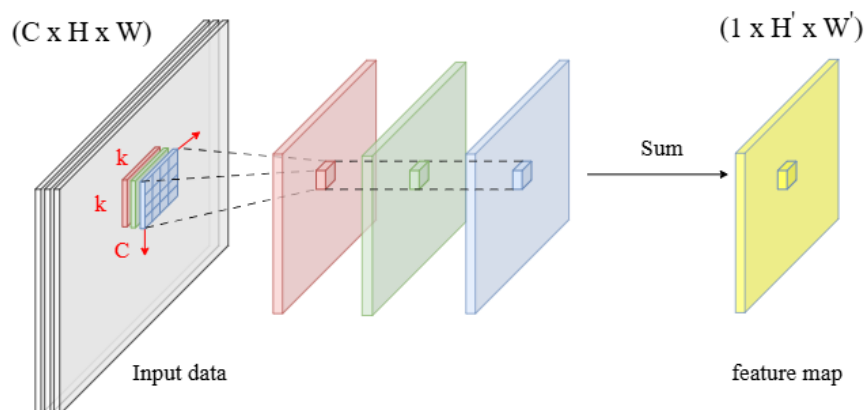
$\gamma, \beta \in \mathbb{R}^C$ (Được khởi tạo $\gamma=1, \beta=0$ trong quá trình huấn luyện mạng sẽ tự học phân phối tối ưu qua hai tham số này)

Vì ConvNeXt sử dụng nn.Linear theo định dạng channel-last, Layer Normalization được áp dụng tự nhiên trên chiều cuối cùng (channel dimension), tương tự cách LayerNorm hoạt động trong Transformer. Đây cũng là lý do ConvNeXt chuyển từ định dạng channel-first sang channel-last trong code thực nghiệm.

3. Inverted Bottleneck với GELU:

- **Pointwise Linear (Expansion $\times 4$):**

Input: $(N, C, H, W) \rightarrow$ Output: $(N, 4C, H, W)$ - Tensor đầu vào được mở rộng số channel lên gấp 4 lần



Hình 8: Minh họa Pointwise Convolution.

Sau khi LN chuẩn hóa phân phối, dữ liệu được đưa vào lớp Pointwise Linear đầu tiên - Phép tích chập 1×1 (hay tương đương: nn.Linear áp dụng tại mỗi pixel). Lớp này học tương tác giữa các kênh mà không trộn thông tin không gian, đồng thời mở rộng số kênh lên 4 lần, đưa dữ liệu vào không gian đặc trưng cao chiều hơn để mạng có thể học các tổ hợp phức tạp hơn giữa các kênh. Bản chất của Pointwise Conv là kernel 1×1 tương đương một Linear layer áp dụng tại từng pixel. Với ảnh RGB ($C=3$), một neuron trong lớp này tính:

$$y = w_1 \cdot R + w_2 \cdot G + w_3 \cdot B + b$$

Để biến đổi từ C kênh đầu vào thành $n = 4C$ kênh đầu ra, ta dùng $4C$ kernel kích thước $C \times 1 \times 1$. Chi phí tính toán:

$$FLOPs_{pw} = C \times 4C \times W \times H = 4C^2 \times H \times W$$

Tổng chi phí cho toàn bộ khối Depthwise Separable Convolution (Depthwise Convolution + Pointwise Convolution):

$$FLOPs_{total} = C \cdot k^2 \cdot H \cdot W + 4C^2 \cdot H \cdot W$$

$$FLOPs_{Total} = (4C^2 \times H \times W) + (C \times k^2 \times H \times W)$$

So sánh với tích chập tiêu chuẩn ($n = 4C$):

$$Tỉ lệ = \frac{k^2 + 4C}{4C \cdot k^2} = \frac{1}{4C} + \frac{1}{k^2}$$

Số tham số lớp Expansion: $C \times 4C$

- **GELU Activation:**

Input: $(N, 4C, H, W) \rightarrow$ Output: $(N, 4C, H, W)$ - Tensor không thay đổi kích thước

Sau khi mở rộng không gian đặc trưng lên $4C$ chiều, hàm kích hoạt GELU được áp dụng để bổ sung tính phi tuyến và chọn lọc mềm các đặc trưng quan trọng - Chuẩn bị cho bước nén về chiều gốc phía sau.

So sánh ReLU và GELU:

ReLU (ResNet): $f(x) = \max(x, 0)$ - Giá trị âm bị loại hoàn toàn, cơ chế "hard gating" (đóng/mở cứng nhắc). Đơn giản, hiệu quả nhưng có thể gây mất thông tin ở vùng giá trị âm.

Mục đích ConvNeXt sử dụng GELU vì ConvNeXt được thiết kế theo hướng hiện đại hóa CNN dựa trên các ý tưởng từ Transformer: Gradient trơn hơn, hạn chế mất gradient, và biểu diễn tốt hơn các đặc trưng phức tạp.

Trong mỗi ConvNeXt block, chỉ sử dụng một hàm kích hoạt GELU được đặt giữa hai lớp Pointwise Linear (Expansion và Projection). Khác với ResNet, nơi các hàm kích hoạt ReLU xuất hiện sau nhiều lớp convolution, ConvNeXt giảm số lượng activation không cần thiết nhằm đơn giản hóa kiến trúc và hạn chế mất mát thông tin do các phép phi tuyến quá nhiều. Thiết kế này được kế thừa từ Transformer block, trong đó activation chỉ xuất hiện tại vị trí trung tâm của khối Feed Forward Network (FFN).

- **Pointwise Linear - (Projection $\times$ 4):**

Input: $(N, 4C, H, W) \rightarrow$ Output: (N, C, H, W) - Nén và tổng hợp lại về chiều gốc

Sau khi GELU chọn lọc đặc trưng trong không gian 4C chiều, lớp Pointwise Linear thứ hai nén và tổng hợp lại về chiều gốc C. Bước này không chỉ giảm kích thước để tiếp tục truyền sang block tiếp theo, mà còn mang toàn bộ những gì mạng đã học trong không gian cao chiều thành một biểu diễn cô đọng hơn.

Số tham số lớp Projection: $4C \times C$

Tổng kết tham số trong ConvNeXt Block (Ví dụ với stage 1):

Thành phần	Tham số
Depthwise Conv 7×7	$96 \times 7 \times 7 = 4,704$
Linear Expansion ($96 \rightarrow 384$)	$96 \times 384 = 36,864$
Linear Projection ($384 \rightarrow 96$)	$384 \times 96 = 36,864$
Layer Scale (γ)	96
Tổng	$\approx 78,528$

Điều này cho thấy phần lớn tham số trong ConvNeXt block tập trung ở hai lớp Pointwise Linear: $73,728 / (4,704 + 73,728) \approx 94\%$. Trong khi Depthwise Convolution chủ yếu thực hiện trích xuất thông tin không gian (spatial information), thì hai lớp Linear mới là thành phần đảm nhiệm phần lớn khả năng biểu diễn và học đặc trưng của block. Sau lớp Pointwise Linear thứ hai, ConvNeXt không sử dụng thêm activation hay normalization mà đưa đầu ra trực tiếp sang Layer Scale và residual

connection. Thiết kế này nhằm giảm số lượng phép biến đổi phi tuyến không cần thiết, giúp thông tin và gradient lan truyền ổn định, trực tiếp hơn giữa các block.

4. Layer Scale và Stochastic Depth:

- **Layer Scale:**

Input: (N, C, H, W) $\rightarrow$ Output: (N, C, H, W)

Tensor đặc trưng có kích thước (N, C, H, W) được đưa qua lớp Layer Scale với hệ số học được $\gamma = 1 \times 10^{-6}$, trong khi kích thước tensor vẫn được giữ nguyên. Lớp này thực hiện việc nhân toàn bộ đầu ra của block với một hệ số rất nhỏ nhằm ổn định quá trình huấn luyện ở giai đoạn đầu và hạn chế việc residual branch làm thay đổi mạnh thông tin gốc, từ đó cải thiện độ ổn định và khả năng hội tụ của mô hình.

Layer Scale thực hiện phép nhân từng channel của tensor đầu ra với một vector hệ số học được γ : $X_{scaled} = \gamma \odot x$, $\gamma \in R^C$, $\gamma_{init} = 10^{-6}$

Trong đó:

$\odot$ là phép nhân element-wise

$\gamma \in R^C$ là vector chứa một hệ số học được cho mỗi channel

Các giá trị γ được khởi tạo rất nhỏ, thường là: $\gamma_{init} = 10^{-6}$ cho tất cả channel.

Ảnh hưởng lên Residual Connection, trong ConvNeXt block, đầu ra cuối cùng được tính theo công thức: $Out = In + \gamma \odot Block(In)$

Ở giai đoạn đầu huấn luyện: $\gamma \approx 0$ nên: $Out \approx In$

Khi đó residual branch gần như bị giảm ảnh hưởng, khiến block hoạt động tương tự một skip connection. Điều này giúp ổn định gradient và hạn chế việc cập nhật quá mạnh ở các tầng sâu. Trong quá trình huấn luyện, các giá trị γ sẽ dần được học và tăng lên nếu cần thiết, cho phép block đóng góp nhiều hơn vào biểu diễn đặc trưng của mô hình.

Vai trò của Layer Scale trong mạng sâu

Khi huấn luyện các mạng rất sâu như ConvNeXt-XL với nhiều residual block liên tiếp, quá trình lan truyền gradient có thể gặp hiện tượng gradient bùng nổ (exploding gradient) hoặc triệt tiêu (vanishing gradient). Layer Scale được sử dụng nhằm ổn định dòng gradient trong quá trình huấn luyện.

- **Stochastic Depth:**

Tensor đặc trưng có kích thước (N, C, H, W) được đưa qua lớp Stochastic Depth trong giai đoạn huấn luyện. Kỹ thuật này thực hiện việc ngẫu nhiên bỏ qua (drop) residual branch của một số block với xác suất nhất định, trong khi kích thước tensor vẫn được giữ nguyên. Trong giai đoạn suy luận, toàn bộ các block đều được sử dụng đầy đủ. Stochastic Depth là một kỹ thuật regularization (kỹ thuật giúp mô hình học tổng quát tốt hơn, tránh việc overfitting) được sử dụng để cải thiện khả năng huấn luyện các mạng rất sâu. Trong quá trình training, mỗi residual block có một xác suất nhất định bị bỏ qua hoàn toàn (skip block), trong khi shortcut connection (đường truyền giúp dữ liệu đi tắt từ đầu vào sang đầu ra của block mà không cần đi qua toàn bộ các lớp bên trong block. Thay vì: $x \rightarrow \text{Block}(x)$, ResNet/ConvNeXt dùng: $x \rightarrow x + \text{Block}(x)$ dữ liệu gốc x được cộng trực tiếp vào đầu ra) vẫn luôn được giữ lại.

Training Phase - Randomly Drop Residual Block:

Với: $b \sim \text{Bernoulli}(1 - p_{\text{drop}})$

Trong đó:

$b = 1$: giữ block

$b = 0$: bỏ qua block

Đầu ra của block được tính: $x_{\text{out}} = b \cdot (\gamma \odot \text{Block}(x_{\text{in}})) + x_{\text{in}}$

Trường hợp block bị skip khi $b = 0$ thì $x_{\text{out}} = x_{\text{in}}$: Khi đó block bị bỏ qua, dữ liệu đầu vào sẽ đi thẳng đến đầu ra thông qua shortcut connection mà không đi qua các lớp xử lý bên trong block.

Trường hợp block được giữ khi $b = 1$ thì $x_{\text{out}} = x_{\text{in}} + \gamma \odot \text{Block}(x_{\text{in}})$: Khi đó block không bị bỏ qua, dữ liệu sẽ đi qua toàn bộ các lớp xử lý trong block như bình thường trước khi được cộng với shortcut connection.

Inference Phase: Toàn bộ block đều hoạt động, không có drop. Không cần scale vì rescaling đã được xử lý ngầm trong training.

Trong Stochastic Depth, mỗi residual block có xác suất p_{drop} bị bỏ qua tạm thời trong quá trình training. Ví dụ, với $p_{\text{drop}} = 0.2$, block sẽ bị skip khoảng 20% số lần huấn luyện và chỉ hoạt động khoảng 80% thời gian. Vì vậy, trong giai đoạn inference, đầu ra residual branch được scale với hệ số $(1 - p_{\text{drop}})$ để giữ độ lớn đặc trưng tương đồng với lúc training. Việc bật/tắt block ngẫu nhiên giúp mô hình học

trên nhiều đường truyền khác nhau, tạo hiệu ứng ensemble và cải thiện khả năng tổng quát hóa. Đồng thời, shortcut connection luôn tồn tại nên dữ liệu và gradient vẫn có thể truyền ổn định ngay cả khi một số block bị bỏ qua. Trong ConvNeXt, xác suất drop của Stochastic Depth không cố định cho tất cả block mà tăng dần theo độ sâu của mạng. Các block ở đầu mạng có xác suất bị drop thấp hơn, trong khi các block ở cuối mạng có xác suất bị drop cao hơn. Cách thiết kế này giúp mô hình duy trì ổn định ở các tầng đầu và tăng khả năng regularization ở các tầng sâu.

$$\text{Xác suất drop: } p_{drop}(l) = p_{max} \times \frac{l}{L-1}$$

Trong đó:

l : chỉ số block

L : tổng số block

p_{max} : xác suất drop lớn nhất

Ví dụ với ConvNeXt-T ($p_{max}=0.1$, tổng 9 block):

Block đầu tiên: $p = 0.0$ (không bị drop)

Block giữa: $p \approx p_{max}/2$

Block cuối: $p = 0.1$

Điều này giúp mạng hoạt động linh hoạt như nhiều mô hình có độ sâu khác nhau trong quá trình huấn luyện.

- **Residual Connection:**

Tensor đầu vào có kích thước (N, C, H, W) được cộng trực tiếp với đầu ra của residual branch thông qua residual connection, tạo ra tensor đầu ra có cùng kích thước (N, C, H, W). Cơ chế này giúp giữ lại thông tin gốc và hỗ trợ gradient lan truyền ổn định trong các mạng sâu.

Đầu ra của một ConvNeXt block được tính toán dựa trên cơ chế residual connection kết hợp với kỹ thuật Layer Scale được biểu diễn như sau:

$$X_{out} = X_{shortcut} + \gamma \cdot f(X)$$

Chú thích:

$X_{shortcut}$: Tensor đầu vào của block (đặc trưng gốc).

$f(X)$: Đầu ra sau toàn bộ các bước xử lý trong block

γ : Tham số học được (đại diện cho Layer Scale) thường có giá trị là 10^{-6}

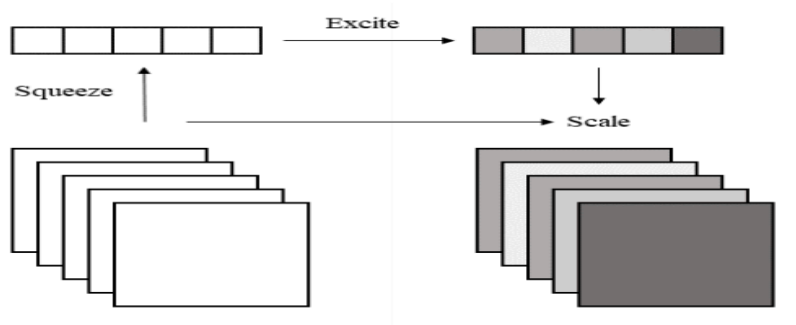
Cách tiếp cận này mang lại hiệu quả tối ưu hóa rõ rệt trong quá trình huấn luyện mạng sâu. Việc khởi tạo γ tiệm cận 0 ở những epoch đầu ép khối hoạt động tương tự một hàm đồng nhất, giúp luồng tín hiệu ổn định trước khi các trọng số của nhánh $f(X)$ kịp thích nghi. Xét trong quá trình lan truyền ngược, phương trình đạo hàm được biểu diễn bằng:

$$\frac{\delta X_{out}}{\delta X_{shortcut}} = 1 + \gamma \cdot \frac{\delta f(X)}{\delta X_{shortcut}}$$

Thành phần hằng số 1 tạo ra một đường truyền trực tiếp cho tín hiệu gradient từ hàm mất mát. Điều này triệt tiêu hiện tượng vanishing gradient, giúp duy trì biên độ gradient ổn định xuyên suốt các tầng kiến trúc. Đặc biệt đối với bài toán nhận diện biểu cảm, cơ chế residual này đảm bảo các đặc trưng không gian nguyên thủy của khuôn mặt từ nhánh $X_{shortcut}$ được bảo toàn, tránh tình trạng xói mòn thông tin khi đi qua hàng loạt phép tích chập, đồng thời dung hợp hiệu quả với các đặc trưng ngữ nghĩa mức cao từ nhánh $f(X)$.

3.4. Cơ chế Squeeze-and-Excitation (SE)

Trong bài toán FER, các vùng khác nhau trên khuôn mặt đóng góp không đồng đều vào việc phân biệt cảm xúc. Do đó SE ra đời như một cơ chế hiệu chỉnh đặc trưng lặp đi lặp lại theo kênh (channel-wise recalibration). Thay vì xử lý tất cả các kênh đặc trưng một cách đồng đều, SE cho phép mạng học cách chủ động cân chỉnh mức độ đóng góp của từng kênh dựa trên nội dung toàn cục của feature map.



Hình 9: A Squeeze-and-Excitation block

Khối SE gồm 2 thành phần xử lý tuần tự: Squeeze và Excitation, sau đó áp dụng kết quả lên feature map thông qua phép Scale.

- **Thành phần Squeeze:**

Nhận feature map đầu vào $X (C \times H \times W)$, bước Squeeze thu gọn toàn bộ thông tin không gian của mỗi kênh thành một giá trị vô hướng duy nhất thông qua Global Average Pooling:

$$z_c = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W x_c(i, j) \quad \text{với } c = 1, \dots, C$$

Kết quả thu được là vector thống kê toàn cục $\mathbf{z} \in \mathbb{R}^C$ có kích thước $(1 \times 1 \times C)$, trong đó mỗi phần tử z_c đại diện cho mức độ kích hoạt trung bình của kênh trên toàn bộ không gian ảnh. Bước này đóng vai trò tổng hợp ngữ cảnh toàn cục - Một thông tin mà các phép tích chập thông thường không thể nắm bắt do bị giới hạn bởi kích thước kernel cục bộ.

- **Thành phần Excitation:**

Excitation nhận vector $\mathbf{z}$ và học một tập trọng số quan trọng theo kênh thông qua một mạng bottleneck hai lớp fully-connected. Để thực hiện điều này hiệu quả, cơ chế Excitation phải đáp ứng hai yêu cầu: (1) có khả năng học được tương tác phi tuyến giữa các kênh, và (2) học được quan hệ cho phép nhiều kênh được nhấn mạnh đồng thời thay vì ép mô hình chỉ chọn một kênh duy nhất. Excitation được định nghĩa như sau:

$$s = \sigma(W_2 \cdot \delta(W_1 \cdot \mathbf{z}))$$

Chú thích:

$W_1 \in \mathbb{R}^{C/r \times C}$: Ma trận chiếu xuống, giảm chiều từ C xuống C/r kênh

$\delta(\cdot)$: Hàm kích hoạt ReLU

$W_2 \in \mathbb{R}^{C \times C/r}$: Ma trận chiếu lên, khôi phục về chiều C kênh

$\sigma(\cdot)$: Hàm Sigmoid, ép output về khoảng $[0,1]$

r : Reduction ratio - Siêu tham số kiểm soát số lượng tham số

Đặc biệt, Thiết kế bottleneck $C \rightarrow C/r \rightarrow C$ giúp giảm số lượng tham số trong khi vẫn học được tương quan liên kênh. Hàm Sigmoid được chọn thay

vì Softmax vì lý do cho phép nhiều kênh được nhấn mạnh đồng thời: Với Softmax, hàm sẽ buộc tổng các trọng số bằng 1, tạo ra cạnh tranh giữa các kênh; Sigmoid cho phép mỗi $S_c \in [0, 1]$ được xác định độc lập, tức là nhiều kênh có thể đồng thời nhận trọng số cao nếu đều mang thông tin hữu ích.

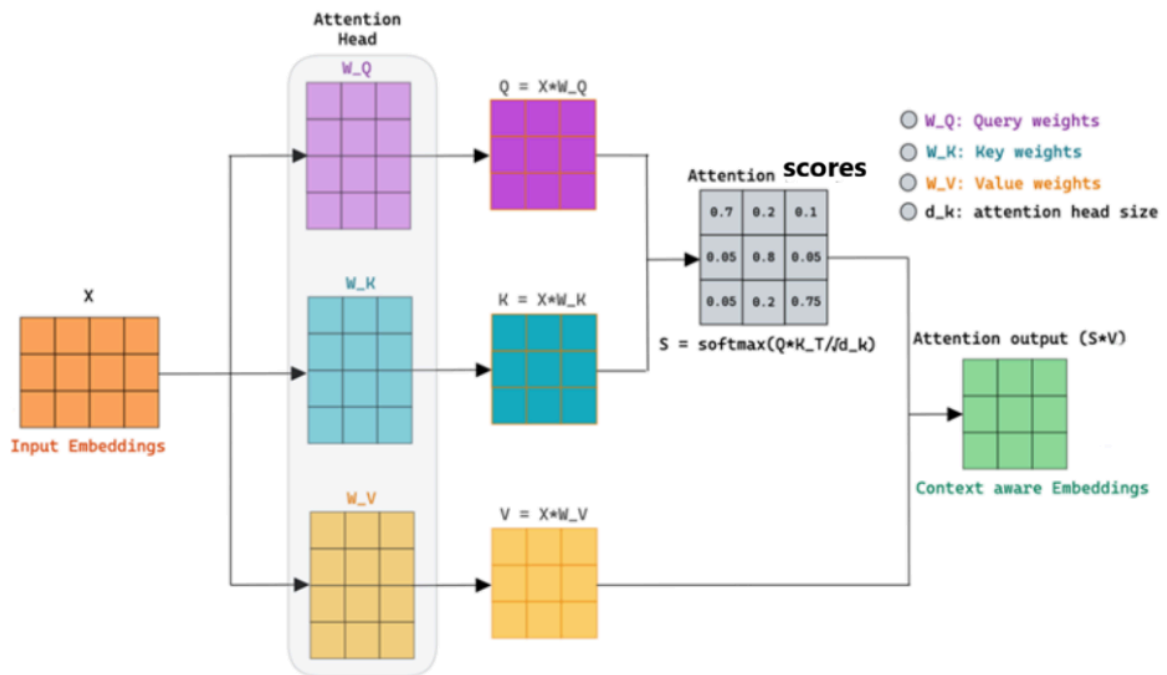
Trong EmoNeXt, một khối SE được chèn sau Downsampling và trước khi feature map được truyền sang stage tiếp theo. Với phiên bản EmoNeXt-Tiny, 4 stages có số chiều kênh lần lượt là $C \in \{96, 192, 384, 768\}$ mỗi stage đi kèm một khối SE với kích thước tương ứng. Việc đặt SE tại ranh giới giữa các stage giúp tái hiệu chuẩn các bản đồ đặc trưng trước khi dữ liệu được chuyển sang giai đoạn tiếp theo. Quá trình chọn lọc và hiệu chỉnh liên tục này giúp mô hình tăng cường khả năng trích xuất các đặc điểm khuôn mặt có tính phân biệt cao, từ đó nhận dạng cảm xúc một cách chính xác hơn.

3.5. Thành phần điều chuẩn tự chú ý (Self-Attention Regularization Term) kết hợp hàm mất mát Cross-Entropy

Trong kiến trúc mạng EmoNeXt, để tối ưu hóa quá trình nhận dạng các biểu cảm khuôn mặt phức tạp, mô hình sử dụng một hàm mất mát tổng hợp. Hàm mất mát này kết hợp chặt chẽ giữa thành phần điều chuẩn tự chú ý (Self-Attention Regularization Term) và hàm Cross-Entropy truyền thống.

Cơ chế Tự chú ý (Self-Attention): Cơ chế Self-Attention được kích hoạt để tìm ra mối liên hệ giữa các vùng trên khuôn mặt. Mỗi vector vùng được biến đổi tuyến tính thành 3 thành phần: Query (Q - Thông tin vùng đó đang tìm kiếm), Key (K - Thông tin các vùng khác cung cấp), và Value (V - Thông tin thực tế). Mức độ liên quan giữa các vùng được tính bằng phép nhân ma trận giữa Q và K^T , chia cho $\sqrt{d}$ (để ổn định gradient), và đưa qua hàm Softmax để chuyển thành phần trăm xác suất. Cơ chế này cho phép mô hình tự học cách chú ý chéo giữa các vùng (ví dụ: vùng mắt tự đối chiếu với vùng miệng) để đưa ra dự đoán biểu cảm chính xác.

$$W(Q, K) = \text{softmax} \left(\frac{Q \cdot K^T}{\sqrt{d}} \right)$$



Hình 10: Cách thức hoạt động của Self-Attention

Softmax đóng vai trò quan trọng trong việc chuẩn hóa các trọng số attention về khoảng $[0,1]$ và đảm bảo tổng bằng 1. Nhờ đó, mô hình có thể tập trung vào những vùng quan trọng hơn trên khuôn mặt. Tuy nhiên, Softmax cũng có xu hướng làm phân phối trở nên “sắc nét”: trọng số trở nên cực đoan, khiến mô hình tập trung quá mức vào các vùng đặc trưng mạnh (như mắt, miệng) và bỏ qua các chi tiết hỗ trợ, dẫn đến hiện tượng quá khớp vào các đặc trưng cục bộ.

$$\frac{e^{Z_i}}{\sum_{j=1}^n e^{Z_j}}$$

Trong đó:

Z_i : Giá trị đầu vào (logits) của lớp thứ i .

e^{Z_i} : Hàm mũ giúp khuếch đại sự khác biệt giữa các giá trị.

$\sum e^{Z_j}$: Tổng các giá trị mũ của tất cả phần tử đầu vào.

n : Số lượng phần tử trong vector.

Từ các trọng số chú ý đã được chuẩn hóa thông qua hàm Softmax, mô hình có thể xác định mức độ quan trọng của từng vùng. Tuy nhiên, sự phân bố này có thể trở nên mất cân bằng nếu không có cơ chế điều chỉnh phù hợp.

Thành phần điều chuẩn (Regularization Term): Nhằm ngăn chặn mô hình bị lệch và chỉ tập trung cục bộ vào một vài điểm ảnh, bài báo đề xuất hàm điều chuẩn $L(W)$ nhằm khuyến khích các trọng số chú ý phân phối đều hơn, tránh việc chỉ tập trung vào một vài điểm ảnh. Công thức được tính như sau:

$$L(W) = \frac{1}{N} \sum_{i=1}^N (W_i - \overline{W})^2$$

Trong đó, N là độ dài của vector đặc trưng và $\overline{W}$ là giá trị trung bình của các trọng số chú ý. Việc đưa giá trị $L(W)$ về mức phù hợp giúp cân bằng lại phân phối của hàm Softmax, ngăn chặn hiện tượng mô hình quá khớp vào một đặc trưng cục bộ, đồng thời buộc mô hình phải học cách phối hợp thông tin từ nhiều vùng khác nhau trên khuôn mặt để dự đoán biểu cảm.

Hàm mất mát tổng hợp (Total Loss): Kết quả cuối cùng được đưa vào hàm mất mát tổng: $L_{final} = L_{CE} + \lambda \cdot L_{SA}$. Trong đó

L_{CE} : Đo lường sai số phân loại giữa dự đoán của mô hình và đáp án gốc

L_{SA} : Hàm phạt $L(W)$

λ : Siêu tham số (hyperparameter) điều chỉnh sự đánh đổi (trade-off) giữa hai thành phần mất mát.

3.6. Các tinh chỉnh của nhóm

Trong quá trình thực nghiệm, thông qua việc đối chiếu chi tiết giữa lý thuyết và mã nguồn, nhóm nhận thấy code gốc của tác giả tồn tại một số điểm chưa nhất quán so với mô tả trong paper được công bố. Những sai lệch này, cùng với một số cấu hình kỹ thuật chưa tối ưu, đã trực tiếp ảnh hưởng đến khả năng hội tụ và hiệu suất của mô hình. Sau khi phân tích kỹ lưỡng các vấn đề, nhóm thực hiện 10 điều chỉnh được tổ chức thành 4 nhóm như trình bày dưới đây.

3.6.1. Sửa lỗi trong cơ chế Self-Attention

Đây là nhóm thay đổi có tác động lớn nhất về mặt kiến trúc, bao gồm 4 điểm liên quan mật thiết với nhau.

Attention scaling không đúng công thức:

Code gốc của tác giả:

$\text{scale} = 1 / \text{math.sqrt}(\text{math.sqrt}(\text{self.input_dim}))$

Code gốc tính hệ số scale theo $1/\sqrt{\sqrt{d}}$ thay vì công thức Scaled Dot-Product Attention chuẩn. Khi d lớn, việc chia không đủ mạnh khiến dot product có phương sai cao, làm softmax bão hòa và gradient bị triệt tiêu.

Nhóm sửa lại thành $\text{scale}=1/\sqrt{d}$ đúng với công thức paper công bố.

Global Average Pooling được áp dụng quá sớm:

Code gốc thực hiện GAP ngay trong `forward_features` trả về vector (B,C) trước khi attention được tính. Hệ quả là attention matrix có kích thước (B,B), tức là mô hình hóa quan hệ giữa các sample trong batch. Lúc này phép tính phụ thuộc vào batch ngẫu nhiên và hoàn toàn vô nghĩa tại thời điểm mô hình suy luận:

```
237  def forward_features(self, x):
238      for i in range(4):
239          x = self.downsample_layers[i](x)
240          x = self.stages[i](x)
241      return self.norm(
242          x.mean([-2, -1])
243      ) # global average pooling, (N, C, H, W) -> (N, C)
...
```

Nhóm đã sửa phương thức `forward_features` này để giữ lại thông tin spatial dimensions giúp attention có thể hoạt động trên 49 spatial tokens, cho phép các vùng khuôn mặt (mắt, miệng, mày) chú ý lẫn nhau, phù hợp với ý tưởng ban đầu của paper. Phương thức sau khi chỉnh trả về feature map (B,C,7,7) trước khi tính GAP:

```
247  def forward_features(self, x):
248      for i in range(4):
249          x = self.downsample_layers[i](x)
250          x = self.stages[i](x)
251      x = x.permute(0, 2, 3, 1) # (B, H, W, C)
252      x = self.norm(x) # LayerNorm trên channel dim
253      x = x.permute(0, 3, 1, 2)
254      return x # (B, C, H, W) <-- giữ lại spatial dimensions để attention hoạt động trên đó
```

Attention output bị bỏ qua: Code gốc tính attention nhưng không sử dụng output vào logit (đầu ra thô) - classification head vẫn nhận feature gốc chưa qua attention:

```

245 ✓ def forward(self, x, labels=None):
246     x = self.stn(x)
247     x = self.forward_features(x)
248     _, weights = self.attention(x)
249     logits = self.head(x)
250
251     if labels is not None:
252         mean_attention_weight = torch.mean(weights)
253         attention_loss = torch.mean((weights - mean_attention_weight) ** 2)
254
255         loss = F.cross_entropy(logits, labels, label_smoothing=0.2) + attention_loss
256         return torch.argmax(logits, dim=1), logits, loss
257
258     return torch.argmax(logits, dim=1), logits
259

```

Sau khi sửa forward_features như ở trên trình bày, nhóm reshape feature map thành $B \times 49 \times C$ (tức là 49 spatial tokens tương ứng với feature map 7×7), áp dụng attention trên các spatial tokens, sau đó lấy trung bình (mean) để thu về (B,C) và đưa vào classification head. Điều này giúp mô hình học được quan hệ giữa các vùng không gian trên khuôn mặt:

```

262 def forward(self, x, labels=None):
263     x = self.stn(x)
264     x = self.forward_features(x) # (B, C, H, W)
265     B, C, H, W = x.shape
266     x = x.flatten(2).transpose(1, 2) # (B, N, C), N = H*W
267     attended_x, weights = self.attention(x) # weights: (B, N, N)
268     x = attended_x.mean(dim=1) # (B, C) global pooling sau attention
269     logits = self.head(x)
270     if labels is not None:
271         mean_attention_weight = weights.mean(dim=-1, keepdim=True) # (B, 49, 1)
272         attention_loss = ((weights - mean_attention_weight) ** 2).mean()
273
274         # loss = F.cross_entropy(logits, labels, label_smoothing=0.2) + attention_loss
275         loss = F.cross_entropy(logits, labels, label_smoothing=0.1) + 0.1 * attention_loss
276         return torch.argmax(logits, dim=1), logits, loss
277     return torch.argmax(logits, dim=1), logits
278

```

SA loss tính mean sai chiều: Dựa trên paper công bố, SA loss được tính:

$$L(W) = \frac{1}{N} \sum_{i=1}^N (W_i - \overline{W})^2$$

Trong đó mean $\overline{W}$ phải được tính per-row - tức mỗi token so sánh với phân phối attention của chính nó. Code gốc tính mean trên toàn bộ tensor thành một scalar duy nhất, không đúng với ý nghĩa của công thức. Nhóm sửa lại thành mean(dim=-1, keepdim=True) để đảm bảo broadcast đúng chiều (Chỗ sửa nằm ở phương thức forward đã sửa ở trên).

3.6.2. Cân chỉnh hàm mất mát

Paper đề xuất hàm mất mát tổng hợp $L_{final} = L_{CE} + \lambda \cdot L_{SA}$ trong đó λ là siêu tham số kiểm soát tỷ trọng giữa hai thành phần. Code gốc cộng trực tiếp hai loss với, trong khi L_{SA} mang giá trị lớn hơn nhiều lần, gây mất cân bằng và làm sai lệch quá trình tối ưu. Nhóm đề xuất điều chỉnh $\lambda = 0.1$ để cân bằng 2 thành phần.

Ngoài ra, label smoothing với hệ số 0.1 được bổ sung vào Cross-Entropy Loss. FER2013 là tập dữ liệu có mức độ nhiễu nhãn cao - Nhiều ảnh mang cảm xúc mơ hồ hoặc bị gán nhãn không nhất quán, do đó label smoothing đóng vai trò regularization phù hợp, ngăn mô hình overfit vào các nhãn không chắc chắn.

Code gốc tác giả:

```
loss = F.cross_entropy(logits, labels, label_smoothing=0.2) + attention_loss
```

Sau khi chỉnh:

```
loss = F.cross_entropy(logits, labels, label_smoothing=0.1) + 0.1 * attention_loss
```

3.6.3. Cải tiến quy trình huấn luyện

Thay thế learning rate scheduler: Code gốc sử dụng Cosine Annealing with Warm Restarts hoạt động theo đơn vị step, với warmup_steps=128, cycle_steps=1024 và max_lr=1e-3.

Giai đoạn warmup trong mỗi cycle được tính:

$$\eta = \eta_{min} + (\eta_{max} - \eta_{min}) \cdot \frac{t_{current}}{T_{warmup}}$$

Giai đoạn cosine decay (sau warmup) được tính:

$$\eta = \eta_{min} + \frac{(\eta_{max} - \eta_{min})}{2} \cdot \left(1 + \cos\left(\pi \cdot \frac{t - T_{warmup}}{T_{cycle} - T_{warmup}}\right) \right)$$

Với mỗi cycle_steps bước, LR lại reset về warmup và lặp lại từ đầu. Vì vậy nên Scheduler này có 3 vấn đề trong ngữ cảnh fine-tuning từ pretrained weights:

1. LR tăng trở lại sau mỗi chu kỳ 1024 steps có thể phá vỡ các biểu diễn đặc trưng đã được học từ ImageNet-22k, đặc biệt ở các layer thấp của ConvNeXt.
2. Với batch size 32 và tương đương 897 steps/epoch trên FER2013, mỗi chu kỳ 1024 steps chỉ kéo dài khoảng 1.14 epoch - tức LR restart gần một lần mỗi epoch, quá thường xuyên để mô hình fine-tune ổn định. .

3. `max_lr=1e-3` là giá trị quá cao cho fine-tuning. Thực tế, contributor trên GitHub repository của tác giả đã phải hạ `max_lr` xuống `1e-5` - giảm 100 lần - mới đạt được kết quả hội tụ.

```
66         self.optimizer = AdamW(model.parameters(), lr=lr)
67         self.scaler = torch.cuda.amp.GradScaler(enabled=self.amp)
68         self.scheduler = CosineAnnealingWithWarmRestartsLR(
69             self.optimizer, warmup_steps=128, cycle_steps=1024
70         )
71         self.ema = EMA(model, beta=ema_decay, update_every=ema_update_every).to(
72             self.device
73         )
```

Paper mô tả sử dụng cosine decay đơn lần. Nhóm thay bằng Linear Warmup 10 epochs kết hợp Cosine Annealing một lần duy nhất với $\eta_{min} = 10^{-7}$, phù hợp hơn với đặc tính fine-tuning: LR tăng dần trong giai đoạn warmup để mô hình thích nghi với phân phối dữ liệu mới, sau đó giảm đơn điệu về η_{min} mà không bao giờ tăng trở lại.

```
77         self.scaler = torch.cuda.amp.GradScaler(self.device.type, enabled=self.amp)
78         steps_per_epoch = len(self.training_data_loader)
79         total_steps = self.epochs * steps_per_epoch
80         warmup = 10 * steps_per_epoch
81         warmup_scheduler = LinearLR(self.optimizer, start_factor=0.1, total_iters=warmup)
82         cosine_scheduler = CosineAnnealingLR(self.optimizer, T_max=total_steps - warmup, eta_min=1e-7)
83         self.scheduler = SequentialLR(
84             self.optimizer,
85             schedulers=[warmup_scheduler, cosine_scheduler],
86             milestones=[warmup]
87         )
```

Tăng early stopping patience: Giá trị mặc định 12 epochs trong code gốc hơi thấp với cosine decay schedule dài, khiến mô hình có thể bị dừng sớm trong giai đoạn loss ngừng giảm và accuracy ngừng tăng tạm thời sau một thời gian hội tụ. Nhóm tăng lên 20 epoch để cho mô hình đủ thời gian vượt qua các giai đoạn cải thiện chậm.

Cập nhật GradScaler API (tính chỉnh nhỏ): API `torch.cuda.amp.GradScaler` đã bị deprecated (trạng thái cảnh báo 1 hàm, không còn được khuyến khích sử dụng) trong các phiên bản PyTorch mới. Nhóm cập nhật sang `torch.cuda.amp.GradScaler` với device type tương minh để đảm bảo tương thích.

3.6.4. Sửa lỗi trong pipeline đánh giá

RandomCrop trong validation: Code gốc áp dụng RandomCrop trong validation transform - Một phương pháp không tối ưu khiến vị trí crop thay đổi ngẫu nhiên mỗi epoch, làm val accuracy dao động và không phản ánh thực chất năng lực của

mô hình. Nhóm thay bằng CenterCrop để đảm bảo validation deterministic và nhất quán.

Batch size của validation loader (tính chỉnh phụ thuộc vào điều kiện phần cứng): Code gốc đặt batch_size=1 cho validation loader, khiến ảnh được forward pass riêng lẻ, không tận dụng GPU parallelism và làm chậm quá trình đánh giá. Nhóm tăng lên batch_size=32 mà không ảnh hưởng đến kết quả.

3.6.5. Ghi chú về tính nhất quán val/test

Trong quá trình thực nghiệm, nhóm nhận thấy một điểm không nhất quán trong thiết kế gốc: checkpoint được lưu dựa trên accuracy của model thường, trong khi test được thực hiện trên EMA model. Hai mô hình này không đồng nhất tại thời điểm lưu checkpoint. Do đó, kết quả test có thể không hoàn toàn tương ứng với val accuracy được dùng để chọn checkpoint. Nhóm ghi nhận đây là hạn chế cần khắc phục trong các thực nghiệm tiếp theo.

CHƯƠNG 4. KẾT QUẢ VÀ THỰC NGHIỆM

4.1. Tập dữ liệu FER2013

4.1.1. Tổng quan và phân chia dữ liệu

FER2013 DATASET SUMMARY

Class	Training	Validation	Testing	Class total
angry	3995	467	491	4953
disgust	436	56	55	547
fear	4097	496	528	5121
happy	7215	895	879	8989
sad	4830	653	594	6077
surprise	3171	415	416	4002
neutral	4965	607	626	6198
Total	28709	3589	3589	35887

Hình 11: Bảng phân phối các lớp của FER2013 Dataset

Tổng quan: FER2013 là bộ dữ liệu nhận diện cảm xúc khuôn mặt được giới thiệu bởi Goodfellow et al. trong khuôn khổ ICML 2013 Challenges in Representation Learning. Bộ dữ liệu gồm 35887 ảnh grayscale kích thước 48×48 pixel, trong đó khuôn mặt được căn chỉnh gần giữa ảnh và chiếm phần lớn diện tích. Mỗi ảnh được gán nhãn thuộc một trong bảy lớp cảm xúc: Angry, Disgust, Fear, Happy, Sad, Surprise và Neutral.

Phân chia dữ liệu: Bộ dữ liệu được chia thành 3 tập cố định: 28709 mẫu huấn luyện (Training), 3589 mẫu xác thực (Validation) và 3589 mẫu kiểm thử (Test). Phân chia này được giữ cố định trong toàn bộ thực nghiệm của nhóm.

4.1.2. Vấn đề mất cân bằng lớp

Dựa trên hình 11 ta có thể thấy FER2013 có phân phối lớp mất cân bằng đáng kể:

Lớp Happy có số mẫu lớn nhất (8989), gấp hơn 16 lần lớp Disgust (547) - Lớp thiểu số nhất trong dataset. Sự mất cân bằng này khiến mô hình có xu hướng bias (thiên lệch) về phía các lớp đa số trong quá trình tối ưu, dẫn đến độ chính xác (accuracy) thấp hơn trên các lớp thiểu số.

Trong thực nghiệm của nhóm, hiện tượng này thể hiện rõ qua kết quả per-class sẽ được trình bày ở phía sau. Mất cân bằng lớp là một trong những nguyên nhân khách quan góp phần tạo ra khoảng cách lớn, bên cạnh độ mơ hồ vốn có của các cảm xúc tiêu cực trên ảnh tĩnh grayscale.

4.2. Cấu hình huấn luyện và Chuyển giao học tập

Toàn bộ thực nghiệm được thực hiện trên kiến trúc EmoNeXt-Tiny, sử dụng pretrained weights từ ConvNeXt-Tiny được huấn luyện trên ImageNet-22k (21.841 lớp). Việc nhóm quyết định lựa chọn pretrained từ ImageNet-22k thay vì ImageNet-1k nhằm tận dụng biểu diễn đặc trưng phong phú hơn từ tập dữ liệu lớn hơn, đồng thời phù hợp với thiết lập thực nghiệm được mô tả trong paper gốc.

Cấu hình huấn luyện:

Siêu tham số (Hyperparameter)	Giá trị
Model size	Tiny
Pretrained	ImageNet-22k
Batch size	32
Gradient accumulation steps	2 (effective batch size = 64)
Learning rate	3e-5
Epochs tối đa	300
Mixed precision (AMP)	True

Scheduler	Linear Warmup + Cosine Annealing
Early stopping patience	20 epoch

Gradient accumulation với 2 steps được sử dụng để đạt effective batch size tương đương 64 trong điều kiện bộ nhớ GPU hạn chế, mà không làm thay đổi động lực tối ưu so với việc train trực tiếp với batch size 64.

Toàn bộ quá trình huấn luyện sử dụng Mixed Precision Training (AMP) để giảm mức tiêu thụ bộ nhớ và tăng tốc độ tính toán.

4.3. Phân tích nguyên nhân và quá trình khắc phục

4.3.1. Hiện tượng mô hình không hội tụ

Khi tiến hành huấn luyện theo đúng hướng dẫn và cấu hình mặc định của tác giả: $lr=1e-4$ với `batch_size=64`, không có gradient accumulation kết quả nhận được: mô hình không hội tụ và val accuracy bị kẹt ở mức 25.1%, tương đương với random guessing trên 7 lớp cảm xúc. Hiện tượng này duy trì ổn định qua nhiều epoch, không có dấu hiệu cải thiện:

```

Eval loss: 1.8674, Eval Accuracy: 24.9373 %
[Epoch: 39/300]
100% | 449/449 [06:53<00:00, 1.08batch/s, loss=1.87, acc=25.1]
wandb: WARNING Artifacts logged anonymously cannot be claimed and expire after 7 days.
Eval loss: 1.8687, Eval Accuracy: 24.9373 %
[Epoch: 40/300]
100% | 449/449 [07:49<00:00, 1.05s/batch, loss=1.87, acc=25.1]
wandb: WARNING Artifacts logged anonymously cannot be claimed and expire after 7 days.
Eval loss: 1.8676, Eval Accuracy: 24.9373 %
[Epoch: 41/300]
100% | 449/449 [07:22<00:00, 1.02batch/s, loss=1.87, acc=25.1]
wandb: WARNING Artifacts logged anonymously cannot be claimed and expire after 7 days.
Eval loss: 1.8679, Eval Accuracy: 24.9373 %
[Epoch: 42/300]
100% | 449/449 [07:59<00:00, 1.07s/batch, loss=1.87, acc=25.1]
wandb: WARNING Artifacts logged anonymously cannot be claimed and expire after 7 days.
Eval loss: 1.8685, Eval Accuracy: 24.9373 %
[Epoch: 43/300]
100% | 449/449 [10:51<00:00, 1.45s/batch, loss=1.87, acc=25.1]
wandb: WARNING Artifacts logged anonymously cannot be claimed and expire after 7 days.
Eval loss: 1.8676, Eval Accuracy: 24.9373 %
[Epoch: 44/300]
100% | 449/449 [12:57<00:00, 1.73s/batch, loss=1.87, acc=25.1]
wandb: WARNING Artifacts logged anonymously cannot be claimed and expire after 7 days.
Eval loss: 1.8679, Eval Accuracy: 24.9373 %
[Epoch: 45/300]
100% | 449/449 [11:33<00:00, 1.55s/batch, loss=1.87, acc=25.1]
wandb: WARNING Artifacts logged anonymously cannot be claimed and expire after 7 days.
Eval loss: 1.8681, Eval Accuracy: 24.9373 %

```

Nhóm nghi ngờ nguyên nhân liên quan đến learning rate. Cụ thể, $lr=1e-4$ là giá trị phù hợp khi huấn luyện từ đầu (training from scratch), nhưng có thể quá lớn đối với bài toán fine-tuning từ pretrained weights. Với learning rate cao, các gradient updates lớn có thể phá vỡ các biểu diễn đặc trưng đã được học từ ImageNet-22k ngay trong những epoch đầu, trước khi mô hình kịp thích nghi với phân phối dữ liệu FER2013. Tuy nhiên, nhóm không có đủ cơ sở thực nghiệm để xác nhận đây là nguyên nhân duy nhất.

4.3.2. Điều chỉnh siêu tham số và Learning Rate (Đạt mức ~67%)

Sau khi tham khảo phần Issues trên repository GitHub của tác giả, nhóm ghi nhận một contributor báo cáo hiện tượng tương tự và đề xuất điều chỉnh: giảm $lr=1e-5$ và đặt $max_lr=1e-5$ trong scheduler. Với cấu hình này, contributor đó đạt được test accuracy 73.6% trên EmoNeXt-Base vẫn không đạt như accuracy trên bản EmoNeXt-Base mà tác giả công bố (74.91%).

Nhóm áp dụng gợi ý trên cho phiên bản Tiny với cấu hình gốc của tác giả ($batch_size=64$ không gradient accumulation). Mô hình hội tụ bình thường và đạt test accuracy là 66.9451%. Kết quả này thấp hơn đáng kể so với con số tác giả báo cáo trong paper (73.34% cho EmoNeXt-Tiny).

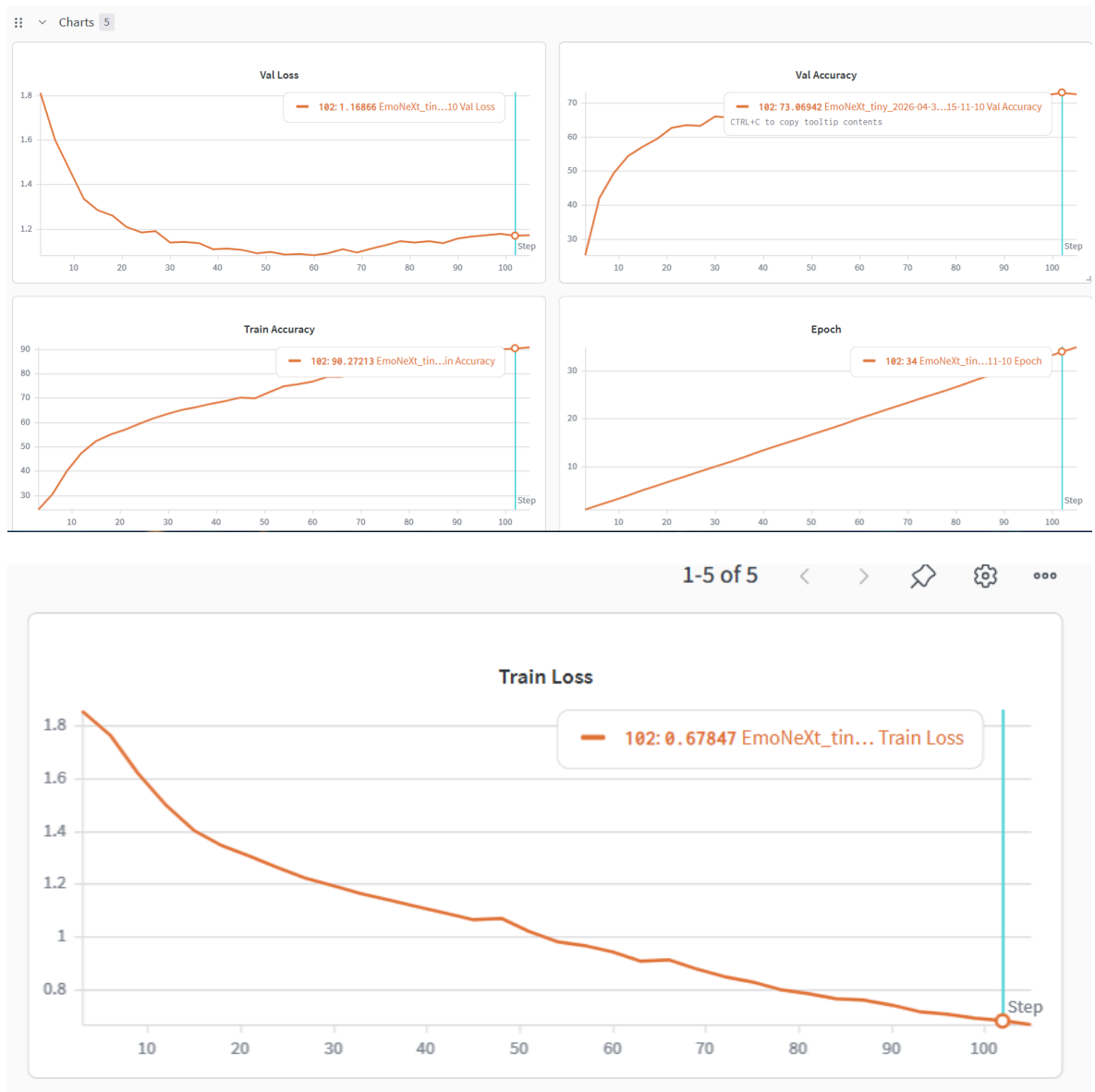
```
Validation loss did not improve for 12 epochs. Stopping training.
100%|
Test Accuracy: 66.9451 %
wandb:
wandb: Run history:
wandb:      Epoch
wandb: Train Accuracy
wandb:   Train Loss
wandb:   Val Accuracy
wandb:    Val Loss
wandb:
wandb: Run summary:
wandb:      Epoch 148
wandb: Train Accuracy 72.86096
wandb:   Train Loss 1.22127
wandb:   Val Accuracy 67.46585
wandb:    Val Loss 1.31735
wandb:
wandb: You can sync this run to the cloud by running:
wandb: wandb sync C:\Users\Wuan Hi Dep Trai\Documents\DataScienceProjects\EmoNeXt\wandb\offline-run-20260419_175400-jkn19cv7
wandb: Find logs at: .\wandb\offline-run-20260419_175400-jkn19cv7\logs
(venv) PS C:\Users\Wuan Hi Dep Trai\Documents\DataScienceProjects\EmoNeXt> |
```

Nhóm nhận định sự chênh lệch này có thể xuất phát từ tổ hợp các thiết kế chưa tối ưu trong code gốc đã được phân tích ở mục 3.6, đặc biệt là attention output bị bỏ qua và SA loss tính mean sai chiều - Những vấn đề này khiến mô hình không học được đúng như thiết kế trong paper, bất kể learning rate được điều chỉnh hợp lý. Đây là động lực trực tiếp dẫn đến quyết định thực hiện các tinh chỉnh được mô tả ở mục 3.6.

4.4. Đánh giá hiệu suất mô hình tinh chỉnh của nhóm

4.4.1. Phân tích kết quả Validation

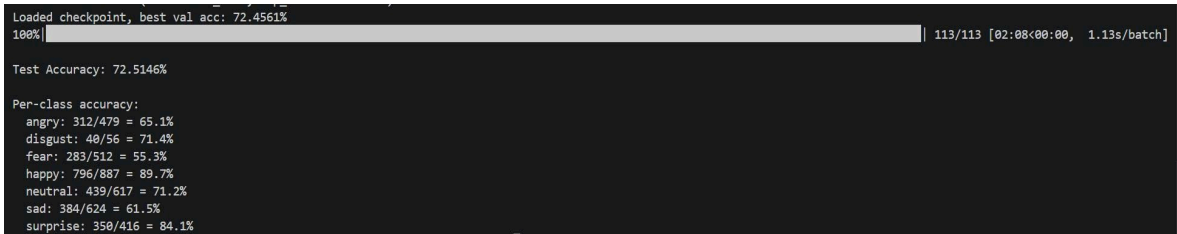
Sau khi áp dụng toàn bộ các cải tiến ở mục 3.6, mô hình được huấn luyện với $lr=3e-5$ và gradient accumulation để đảm bảo ổn định trong fine-tuning. Trong quá trình theo dõi, val accuracy cao nhất quan sát được là **73.06942%**, tuy nhiên giá trị này không được lưu lại do cơ chế checkpoint của hệ thống lưu model dựa trên model thường trong khi EMA model chưa được đồng bộ tại thời điểm đó.



Checkpoint tốt nhất được lưu thành công tương ứng với **val accuracy 72.4561%**. Nhìn chung, quá trình huấn luyện cho thấy mô hình hội tụ ổn định và cải thiện đều đặn so với baseline 66.9451% đạt được ở giai đoạn trước, xác nhận tác động tích cực của các điều chỉnh kiến trúc và quy trình huấn luyện.

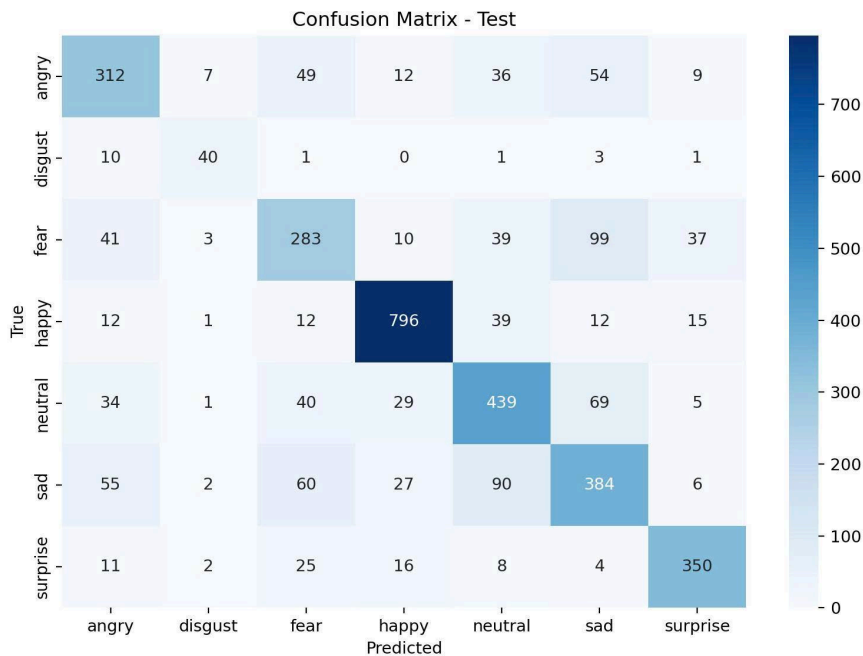
4.4.2. Kết quả đánh giá trên tập Test

Checkpoint val 72.4561% được đưa vào đánh giá trên tập test sử dụng EMA model. Kết quả đạt được là **test accuracy 72.5416%** và độ chính xác trên từng lớp:



Hình 11: Kết quả đánh giá trên tập Test

Phân tích kết quả theo từng lớp dựa trên confusion matrix:



Hình 12: Ma trận nhầm lẫn trên tập Test

Từ ma trận trên, ta tính được độ chính xác cho từng lớp bằng cách lấy số mẫu dự đoán đúng (đường chéo chính) chia cho tổng số mẫu thực tế của lớp đó:

Angry	Disgust	Fear	Happy	Sad	Surprise	Neutral
65.1%	71.4%	55.3%	89.7%	61.5%	84.1%	71.2%

- **Nhóm 1 - Phân loại tốt (>80%):**

Happy (89.7%) và Surprise (84.1%) là 2 lớp mô hình xử lý tốt nhất. Lý do khách quan khá rõ: đây là hai cảm xúc có đặc trưng hình thái rõ ràng và ít mơ hồ. Happy đặc trưng bởi miệng mở/cười và vùng má nâng cao; Surprise đặc trưng bởi mắt mở

to và miệng há. Hai cảm xúc này cũng ít bị nhầm lẫn với nhau và ít chồng lấp với các cảm xúc tiêu cực. Ngoài ra, Happy là lớp có số lượng mẫu lớn nhất trong FER2013 (8989 mẫu), giúp mô hình học được biểu diễn đầy đủ hơn.

- **Nhóm 2 - Phân loại trung bình khá (65–75%):**

Disgust (71.4%) và Neutral (71.2%) đạt mức trung bình khá.

Disgust là lớp đáng chú ý vì đây lớp này có số mẫu ít nhất trong toàn bộ dataset. Với 71.4% trên chỉ 56 mẫu test, kết quả có thể dao động lớn và không đủ ổn định để đánh giá chắc chắn. Về mặt hình thái, Disgust thường biểu hiện qua vùng mũi nhăn môi trề, thậm chí là nhú mắt - Các đặc trưng tinh tế hơn Happy hay Surprise, nên dễ bị nhầm với Angry.

Neutral thường bị nhầm với Sad và Fear nhất, vì chủ yếu do sự tương đồng về đặc điểm thị giác (khóe miệng trề, cơ mặt thả lỏng) và cường độ biểu cảm thấp khiến ranh giới đặc trưng giữa trạng thái bình thường và cảm xúc tiêu cực trở nên mơ hồ.

Angry (65.1%) nằm ở ranh giới dưới của nhóm này. Angry thường bị nhầm với Fear và Sad xuất phát từ sự tương đồng của các tổ hợp đặc trưng vùng chân mày và miệng trong các trạng thái tiêu cực, kết hợp với nhiễu về ánh sáng và góc chụp trên tập FER2013 làm mờ đi các ranh giới phân loại của mô hình.

- **Nhóm 3 - Phân loại kém (<65%):**

Sad (61.5%) và Fear (55.3%) là hai lớp mô hình xử lý kém nhất, và đây là điểm yếu rõ ràng nhất.

Fear (55.3%) gần như ngang mức random guessing cho một bài toán nhị phân - là kết quả đáng lo ngại nhất. Lý do khách quan: Fear là cảm xúc có biểu hiện hình thái gần giống nhất với Surprise (mắt mở to, lông mày nâng), Neutral và Sad nhưng khác ở sắc thái tinh tế hơn như vùng miệng và căng thẳng cơ mặt. Điều này cho thấy mô hình chưa thể học được các đặc trưng phân biệt tinh tế cho Fear, một phần do ảnh tĩnh grayscale không cung cấp đủ thông tin.. Kết quả này củng cố nhận định rằng hạn chế này đến từ chất lượng dữ liệu nhiều hơn là từ thiết kế mô hình.

Sad (61.5%) bị nhầm nhiều với Fear và Neutral. Sad thường biểu hiện qua khóe miệng hướng xuống và vùng mắt buồn - Các đặc trưng có thể rất tinh tế. Trong tập FER2013, nhiều ảnh nhân Sad chỉ thể hiện sự buồn nhẹ, khiến khóe miệng và cơ mặt không có sự biến dạng đủ lớn để phân biệt với khuôn mặt bình thường. Với cả Sad và Fear đều thường xuất hiện với đặc điểm chân mày hơi nhướng lên hoặc co lại ở phía trong. Nếu cường độ biểu cảm không đủ mạnh để làm mở to mắt (đặc trưng của Fear), mô hình EmoNeXt dễ bị nhầm lẫn giữa hai trạng thái tiêu cực này.

Đánh giá tổng thể mô hình dựa trên kết quả phân tích:

- **Mất cân bằng lớp chưa được xử lý triệt để:** Khoảng cách giữa Happy (89.7%) và Fear (55.3%) là 34.4%, một khoảng cách rất lớn cho thấy mô hình bị bias rõ ràng về phía các lớp có nhiều mẫu. Nhóm chưa áp dụng class weighting hay oversampling cho các lớp thiểu số.
- **Fear và Sad là hai điểm yếu model:** Hai lớp này chiếm 31% trong dataset nhưng đều có accuracy dưới 62%. Nếu mô hình được triển khai thực tế, đây là hai cảm xúc sẽ bị nhận diện sai thường xuyên nhất.
- **Giới hạn của dữ liệu FER2013:** Về thông tin màu sắc (kênh màu) và mang tính chất là ảnh tĩnh, thiếu thông tin thời gian, nên đặc biệt bất lợi cho Fear và Sad vốn phụ thuộc nhiều vào chuyển động và sắc thái biểu hiện tinh tế theo thời gian.

4.4.3. So sánh định lượng với các mô hình SOTA

Để đánh giá hiệu quả của EmoNeXt sau khi thực hiện các cải tiến, nhóm tiến hành so sánh kết quả đạt được của mô hình EmoNeXt-Tiny trên tập dữ liệu FER2013 với các kiến trúc tiên tiến khác đã được công bố.

Model	Accuracy (%)	Ghi chú
ResNet50	73.20	
VGG	73.28	
CNN + BOVW	75.42	
LHC-NetC	74.28	
Segmentation VGG-19	75.97	
ConvNeXt-Base	73.22	
ConvNeXt-Tiny	71.99	
EmoNeXt-Tiny (gốc)	73.34	Công bố của tác giả
EmoNeXt-Tiny (khởi tạo lại LR)	66.95	Điều chỉnh LR
EmoNeXt-Tiny (sau tinh chỉnh)	72.55	Nhóm thực hiện

Kết quả sau cải tiến của nhóm (72.54%) thấp hơn 0.8 điểm phần trăm so với công bố gốc của tác giả (73.34%) [1]. Ở mức 72.55%, EmoNeXt-Tiny sau cải tiến vượt qua ResNet50 (73.20%) và VGG (73.28%) - Thành tựu đáng ghi nhận bởi EmoNeXt-Tiny có số lượng tham số khiêm tốn hơn đáng kể so với các mô hình này. Tuy nhiên, kết quả này vẫn thấp hơn các mô hình phức tạp hơn như LHC-NetC (74.28%) và đặc biệt là Segmentation VGG-19 (75.97%). Điều này cho thấy Lợi thế của các mô hình chuyên biệt: LHC-NetC được thiết kế đặc biệt với cơ chế self-attention kênh cho FER, trong khi Segmentation VGG-19 tích hợp U-Net để nhấn mạnh các vùng đặc trưng.

Hạn chế của kiến trúc thuần CNN: Mặc dù ConvNeXt được cải tiến từ các ý tưởng của Transformer, việc loại bỏ hoàn toàn cơ chế attention trên ảnh có thể là một hạn chế so với các mô hình hybrid.

Điều đáng chú ý nhất trong bảng so sánh là khoảng cách giữa kết quả code gốc và LR điều chỉnh (66.95%) và kết quả sau cải tiến (72.54%). Khoảng cách 5.59% này phản ánh chính xác giá trị mà các phát hiện và điều chỉnh của nhóm đóng góp. Nói cách khác, nếu chỉ điều chỉnh learning rate (như gợi ý của contributor), mô hình chỉ đạt 66.95% - Thấp hơn đáng kể so với công bố. Chỉ sau khi sửa các lỗi kiến trúc (Mục 3.6.1) và cân chỉnh hàm mất mát (Mục 3.6.2), mô hình mới tiến gần đến mức công bố của tác giả.

CHƯƠNG 5. KẾT LUẬN

5.1. Tóm tắt hiệu quả của EmoNeXt

- **Hiệu suất vượt trội (trên tập dữ liệu FER2013):**

Độ chính xác: Biến thể EmoNeXt-XLarge đạt 76.12%, thiết lập tiêu chuẩn mới so với Segmentation VGG-19 (75.97%) và LHC-NetC (74.28%). Và ngay cả bản Tiny (73.34%) cũng cho hiệu suất tương đương các kiến trúc lớn như ResNet50.

Tối ưu hóa tài nguyên: Với 74.91%, EmoNeXt-Base là lựa chọn cân bằng nhất. Mô hình cải thiện 1.57% độ chính xác so với Tiny nhưng vẫn đảm bảo khả năng huấn luyện trên GPU tầm trung và hội tụ nhanh, phù hợp cho các tác vụ fine-tuning.

- **Các cải tiến kỹ thuật cốt lõi:**

Cơ chế thích nghi không gian và kênh: Việc tích hợp STN và SE Blocks giúp mô hình tự động xử lý các biến dạng khuôn mặt và lọc đặc trưng kênh quan trọng, loại bỏ các bước tiền xử lý thủ công phức tạp.

Regularization bằng Self-Attention: Sử dụng số hạng điều hòa tự chú ý để kiểm soát trọng số học tập, ngăn chặn hiện tượng quá khớp và đảm bảo mô hình tập trung đồng đều vào các vùng đặc trưng biểu cảm quan trọng.

Tính linh hoạt: Hệ thống cung cấp dải biến thể rộng (từ Tiny đến XLarge), cho phép triển khai linh hoạt trên nhiều thiết bị từ cấu hình thấp đến các hệ thống tính toán hiệu năng cao.

5.2. Hướng phát triển trong tương lai

Dựa trên các kết quả đạt được và những hạn chế đã phân tích, nhóm đề xuất một số hướng phát triển nhằm cải thiện EmoNeXt trong các nghiên cứu tiếp theo trong tương lai:

Kiến trúc EmoNeXt hiện tại sử dụng số hạng điều chuẩn tự chú ý (Self-Attention Regularization) để khuyến khích trọng số attention trải đều trên các chiều đặc trưng, nhằm tránh phụ thuộc quá mức vào một tập con đặc trưng. Tuy nhiên, trong thực tế, mỗi cảm xúc thường liên quan đến các vùng khuôn mặt khác nhau (Vui → miệng, Ngạc nhiên → mắt). Việc ép dàn trải đồng đều có thể làm mô hình phân tán sự chú ý vào các vùng ít liên quan. Hướng phát triển là thiết kế cơ chế attention phân biệt theo từng lớp cảm xúc. Trong đó, mỗi lớp cảm xúc học một phân phối attention riêng. Cách tiếp cận này vừa giữ được lợi ích regularization, vừa cho phép mô hình tập trung đúng vào các vùng đặc trưng cho từng biểu cảm.

Xử lý vấn đề mất cân bằng lớp dữ liệu. Khoảng cách độ chính xác giữa lớp nhiều mẫu nhất (Happy: 89.7%) và lớp ít mẫu nhất (Disgust: 71.4%) phản ánh rõ tác động của mất cân bằng dữ liệu. Các giải pháp đề xuất bao gồm: (i) trọng số mất mát theo lớp (class-weighted loss) để tăng trọng số cho các lớp thiểu số; (ii) kỹ thuật lấy mẫu nâng cao kết hợp oversampling và undersampling; (iii) augmentation đặc thù cho các lớp ít mẫu như Disgust và Fear.

Khắc phục giới hạn của dữ liệu tĩnh và ảnh xám. Hạn chế của FER2013 (ảnh tĩnh, grayscale) khiến mô hình khó phân biệt các cảm xúc tinh tế như Fear và Sad, vốn được nhận diện tốt hơn qua chuyển động khuôn mặt. Hướng phát triển là mở rộng EmoNeXt sang dữ liệu video bằng cách tích hợp module xử lý chuỗi thời gian như để học thông tin về cường độ và tốc độ thay đổi biểu cảm. Hoặc đánh giá trên đa dạng tập dữ liệu và ứng dụng thực tế. Cần mở rộng thực nghiệm trên các tập dữ liệu khác để đánh giá khả năng tổng quát hóa.